



Storage classes and program structure

(+ intro to pointers)

Lecture overview

- Function declarations and definitions – recap ←
- Variable scope and storage classes
- Program memory segments
- Examples of program execution
- Introduction to pointers

Functions in C – recap

C is a procedural programming language, so the basic building blocks of each program are functions

```
int sum(int, int);
```

← declaration

```
int main() {  
    double a,b;  
    scanf("%lf%lf", &a, &b); // %lf = long float (double)  
    sum(a,b);  
    return 0;  
}
```

← invocation

```
int sum(int a, int b) {  
    printf ("%d+%d=%d\n", a, b, a+b);  
    return a+b;  
}
```

} ← declaration

} ← definition

Functions in C – recap

C is a procedural programming language, so the basic building blocks of each program are functions

- *Function declaration*: specifies to compiler parameter types and return type
- *Function definition*: actual code that describes what the function does
 - Each function should **be defined exactly once**, but it can be **declared several times** (the definition contains an implicit declaration)
 - No function name overriding in C
 - The declaration(s) and definition should be 100% type consistent
 - Invocations do not have to be type consistent
 - **type conversions** are allowed

Lecture overview

- Function declarations and definitions – recap
- Variable scope and storage classes ←
- Program memory segments
- Examples of program execution
- Introduction to pointers

Variable: scope vs. life span

Each variable is defined by two aspects:

- **Scope** – the section of code the variable is relevant for
- **Life span** – the time within the execution of the program that the variable is alive

Local variable scope

Nearly all variables in your programs will be **locally defined** in a function, or a code block within a function (e.g., within a loop)

- The scope of a local variable is confined to the function (or code block) it is defined in
- Scope propagates (“moves downward”) to include all code blocks within the function (or code block) the function is defined in
- **Variables with the same identifier (name) can be defined in other functions (or code blocks), even with different type**

Local variable scope – example 1

- Code blocks can be part of **for** loops, **if** statements, etc.
- Code blocks can also be defined ad-hoc:

```
int main() {  
    int a=5, b=3;                                ← a,b local in main()  
    printf("%d / %d = %d\n", a,b, a/b);  
    {  
        double a=7;                               ← a local in block  
        printf("%.11f / %d = %.11f\n", a,b,a/b);    ← b propagates into block  
    }  
    printf("%d / %d = %d\n", a,b, a/b);  
    return 0;  
}
```

5 / 3 = 1

7.0 / 3 = 2.3

5 / 3 = 1

Local variable scope – example 2

```
int sum(int a, int b) {  
    return a+b;  
}
```

← local in **sum()**

```
int main() {  
    int i, res;  
    for(i=0; i<2; i++) {  
        int a, b, res;  
        scanf("%d%d", &a, &b);  
        res = sum(a,b);  
        printf("%d + %d = %d\n", a,b,res);  
    }  
  
    return 0;  
}
```

← local in **main()**

← local in **for** loop

code in `/share/classes/class07/sum.c`

```
20 10  
20 + 10 = 30  
2 3  
2 + 3 = 5
```

Local variable scope – example 2

```
int sum(int a, int b) {  
    return a+b;  
}
```

← local in **sum()**

```
int main() {  
    int i, res;  
    for(i=0; i<2; i++) {  
        int a, b, res;  
        scanf("%d%d", &a, &b);  
        res = sum(a,b);  
        printf("%d + %d = %d\n", a,b,res);  
    }  
    printf("res = %d\n",res);  
    return 0;  
}
```

← local in **main()**

← local in **for** loop

← **main's** variable
(uninitialized value)

Unlike Java!

code in `/share/classes/class07/sum.c`

20 10

20 + 10 = 30

2 3

2 + 3 = 5

res = 32766 ← “garbage” value that is written in the bits of the uninitialized **res**

Local variable scope – example 2

```
double div(double a, double b) {  
    return a/b;  
}  
  
int main() {  
    int i, res;  
    for(i=0; i<2; i++) {  
        int a, b; double res;  
        scanf("%d%d", &a, &b);  
        res = div(a,b);  
        printf("%d / %d = %lf\n", a,b,res);  
    }  
    printf("res = %d\n",res);  
    return 0;  
}
```

← local in div()

← local in main()

← local in for loop

← main's variable
(uninitialized value)

code in `/share/classes/class07/div.c`

- Variables with the same identifier (name) can be defined in other functions (or code blocks), even with different type

Global variables

- If you wish to define a variable that can be accessed by several functions you may **globally define** it outside of any function
- The scope of a global variable includes all functions defined after its definition

```
void set (int);  
int get (void);  
  
int main() {  
    set (100);  
    printf ("%d\n", get());  
    return 0;  
}  
  
int global;  
  
void set (int n) {  
    global = n;  
}  
  
int get() {  
    return global;  
}
```

Variable life span

Local variables:

- A local variable is **alive** as long as its function is active
- It is automatically allocated every time a function is called and freed when the function returns

Global variables:

- A global variable is **alive** throughout the run time of the program

Local vars are **not initialized** (unless explicitly assigned a value)

Global vars are **initialized to 0** (unless explicitly assigned a different value)

Variable scope and life span – summary

Local (auto) variable:

- scope confined to current code block (and all blocks contained in it)
- life span confined to current invocation of code (re-allocated every time block is entered)

Global (extern) variable:

- scope from definition point to end of code file
- life span is entire run time of program

Introducing: Static variables (`static`)

Local (`auto`) variable:

- scope confined to current code block (and all blocks contained in it)
- life span confined to current invocation of code (re-allocated every time block is entered)

Global (`extern`) variable:

- scope from definition point to end of code file
- life span is entire run time of program

Static (`static`) local variable:

- scope confined to current code block (and all blocks contained in it)
- life span is entire run time of program

Note: `static` in Java has a different meaning than in C!

Unlike Java!

Static variables

- Normally, when a variable is defined within a function, it is **re-allocated every call**
- A **static** variable is assigned a memory location at **compile time** (before the program starts to run), and **retains its value between subsequent calls**
- Example:

```
#include <stdio.h>

int f() {
    static int count = 0;
    return (count++);
}

int main() {
    printf ("%d ", f());
    printf ("%d ", f());
    printf ("%d \n", f());
    return 0;
}
```

← **Variable definition and initialization applied once before all invocations of function. Function execution will not contain this statement!**

Summary: Local, Global, Static

Local (auto) variable:

- scope confined to current code block (and all blocks contained in it)
- life span confined to current invocation of code (re-allocated every time block is entered)

Global (extern) variable:

- scope from definition point to end of code file
- life span is entire run time of program

Static (static) local variable:

- scope confined to current code block (and all blocks contained in it)
- life span is entire run time of program

Online poll

What does the following program print?

```
#include <stdio.h>

int num = 1;

int f1() {
    int num = 100;
    num++;
    return num;
}

int f2() {
    num++;
    return num;
}

int f3() {
    static int num = 10;
    num++;
    return num;
}
```

```
int main() {
    int i;
    for(i=0; i<3; i++) {
        printf("%2d: f1() returns %3d\n", i, f1());
        printf("%2d: f2() returns %3d\n", i, f2());
        printf("%2d: f3() returns %3d\n", i, f3());
    }
    return 0;
}
```

Online poll – solutions

What does the following program print?

```

#include <stdio.h>

int num = 1;

int f1() {
    int num = 100;
    num++;
    return num;
}

int f2() {
    num++;
    return num;
}

int f3() {
    static int num = 10;
    num++;
    return num;
}
  
```

```

int main() {
    int i;
    for(i=0; i<3; i++) {
        printf("%2d: f1() returns %3d\n", i, f1());
        printf("%2d: f2() returns %3d\n", i, f2());
        printf("%2d: f3() returns %3d\n", i, f3());
    }
    return 0;
}
  
```

```

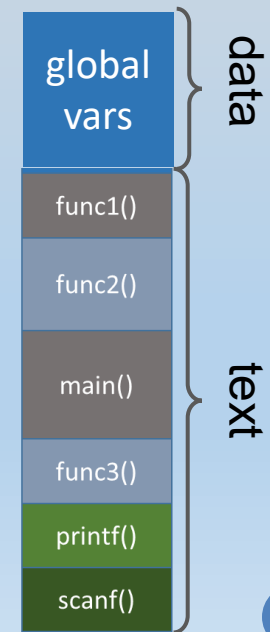
0: f1() returns 101 ← f1() returns the unchanged
0: f2() returns 2   value of local num=101
0: f3() returns 11
1: f1() returns 101
1: f2() returns 3   ← f2() updates and returns
1: f3() returns 12  value of global var
2: f1() returns 101
2: f2() returns 4
2: f3() returns 13 ← f3() updates and returns
                    value of (local) static var
  
```

Lecture overview

- Function declarations and definitions – recap
- Variable scope and storage classes
- Program memory segments ←
- Examples of program execution
- Introduction to pointers

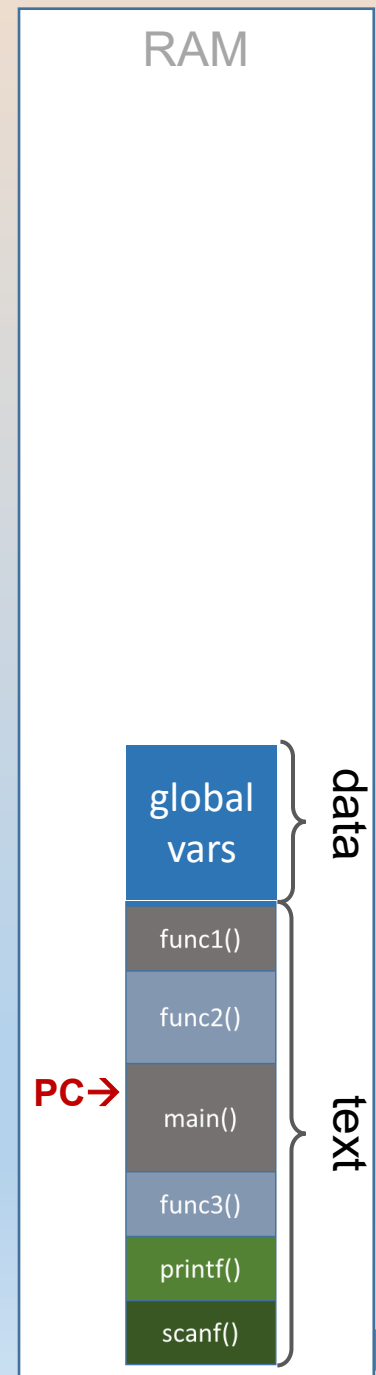
Structure of the compiled executable

- The compiler takes source code in a programming language (C, Java, etc.) and translates it to **machine code**
- Each function is separately translated to machine language, by taking the “high level” **statements** and converting them to **instructions** machine code
- The binary executable file generated by the compiler contains a **block** for each function, which contains the **instructions** and the **storage space** for all local vars (pre-prepared “frames” that are later loaded to the call stack)
- The **data segment** is a separate block allocated for the global (and local static) variables. Their initial values are already set at compilation



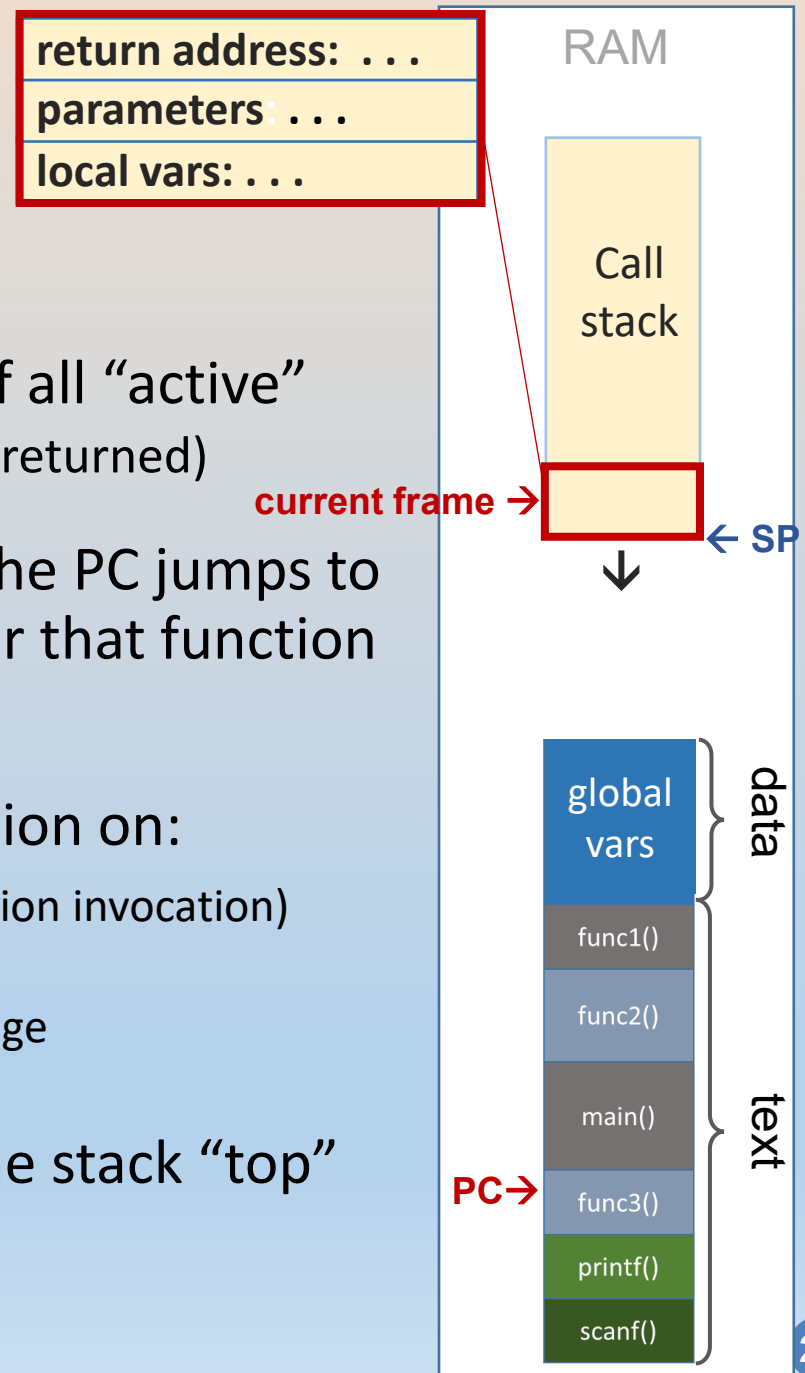
The program counter (PC)

- When a program is executed, its entire code (text and data segments) is loaded to memory (RAM)
- During program execution, a **program counter (PC)** is used to indicate the next instruction in the **text segment** that the machine needs to perform
- The PC typically moves to the next instruction, unless encountering one of the following:
 - Conditional statement (if / case / conditional assignment)
 - Loop (for / while)
 - Function call / return



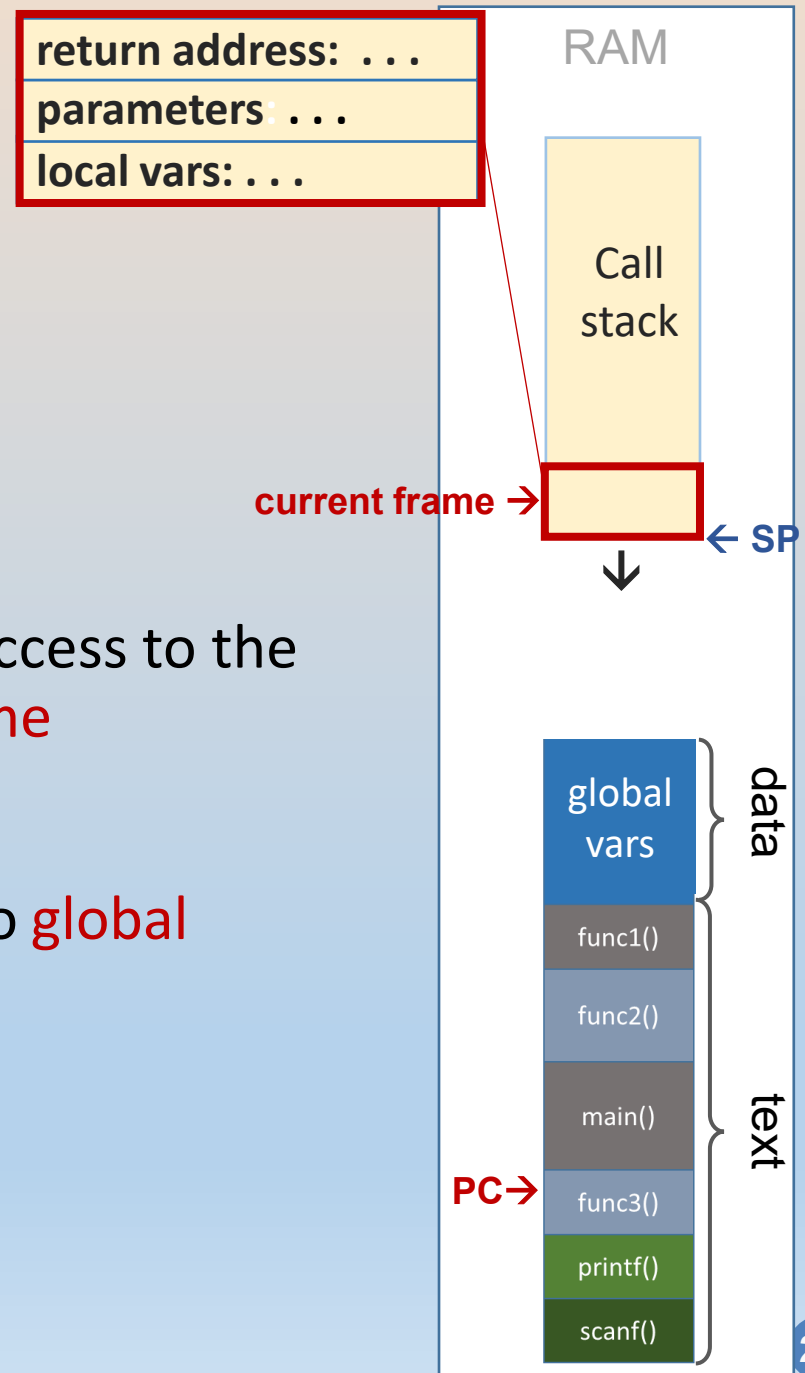
The call stack

- The **call stack** saves information of all “active” functions (those invoked but not yet returned)
- Every time a function is invoked, the PC jumps to that function and a **stack frame** for that function is added to the call stack
- The stack frame contains information on:
 - Return address (location of PC at function invocation)
 - Function parameters
 - Local variables
- The **stack pointer** (SP) indicates the stack “top”



Storage classes

- At each point, the program has access to the **local variables** of the **current frame**
- The program always has access to **global variables** in the **data segment**



Lecture overview

- Function declarations and definitions – recap
- Variable scope and storage classes
- Program memory segments
- Examples of program execution ←
- Introduction to pointers

Program execution example #1

What does the execution of this program look like?

```
#include <stdio.h>

int num = 1;

int f1() {
    int num = 100;
    num++;
    return num;
}

int f2() {
    num++;
    return num;
}

int f3() {
    static int num = 10;
    num++;
    return num;
}
```

```
int main() {
    int i;
    for(i=0; i<3; i++) {
        printf("%2d: f1() returns %3d\n", i, f1());
        printf("%2d: f2() returns %3d\n", i, f2());
        printf("%2d: f3() returns %3d\n", i, f3());
    }
    return 0;
}
```

code in `/share/classes/class07/varscopes.c`

```

int main() {
    int i;
    for(i=0; i<3; i++) {
        printf("%2d: f1() returns %3d\n", i, f1());
        printf("%2d: f2() returns %3d\n", i, f2());
        printf("%2d: f3() returns %3d\n", i, f3());
    }
    return 0;
}
    
```

Program execution example #1

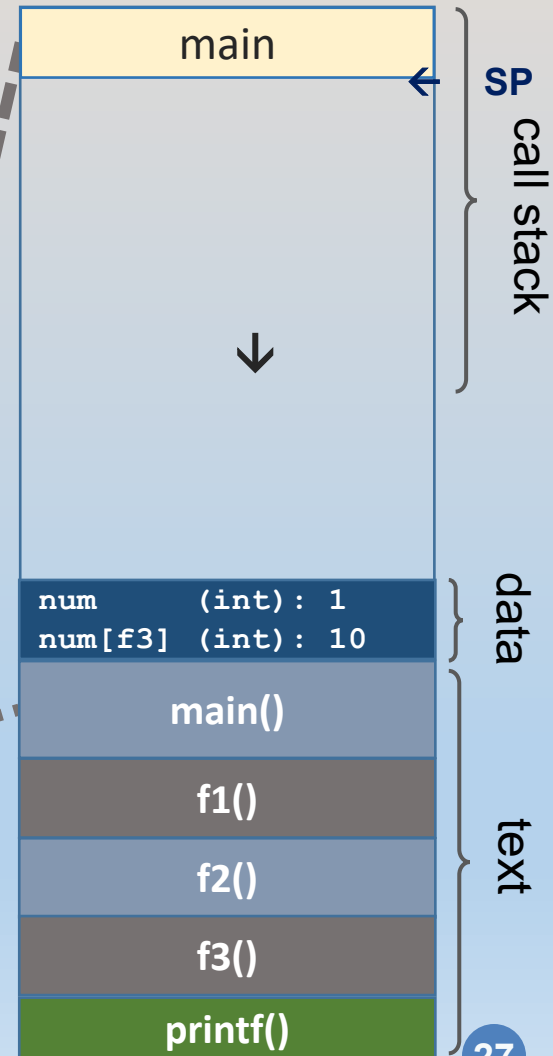
- Program loads to text segment of RAM
- Global / static data are initialized

- `int num = 1;`
- `int f1() {`
`int num = 100;`
`num++;`
`return num;`
`}`
- `int f2() {`
`num++;`
`return num;`
`}`
- `int f3() {`
`static int num = 10;`
`num++;`
`return num;`
`}`

return address: - system -

parameters: - none -

local vars: i(int): 0



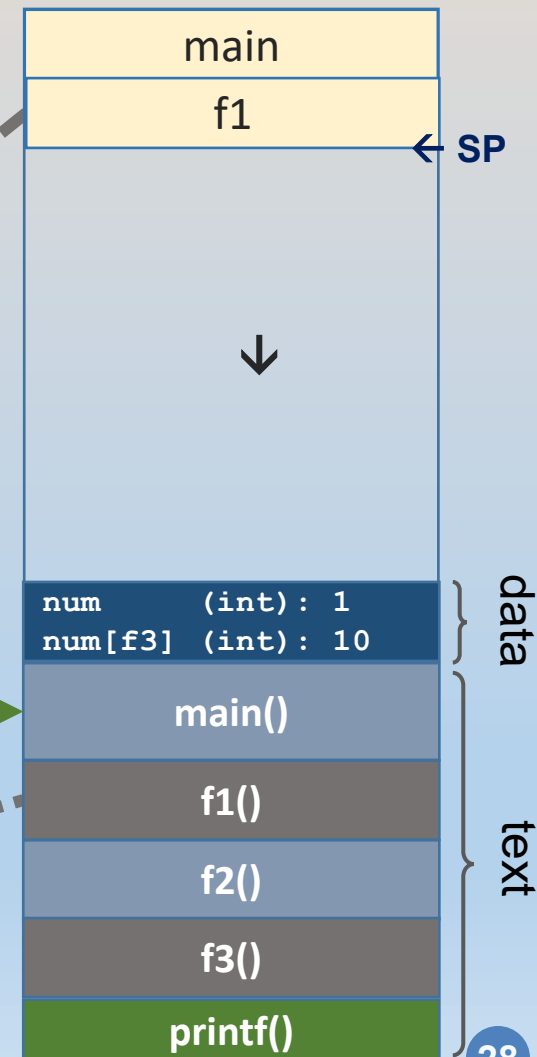
Program execution example #1

- Function `f1()` is invoked and its stack frame is pushed
- PC moves to the appropriate code block
- Return address (in `main`) is saved in stack frame
- Variable `num` accessed in `num++` operation is local var (in stack frame)

return address:	(in main)
parameters:	- none -
local vars:	num (int): 100

```

f1:
PC→ num++;
    return num;
    
```



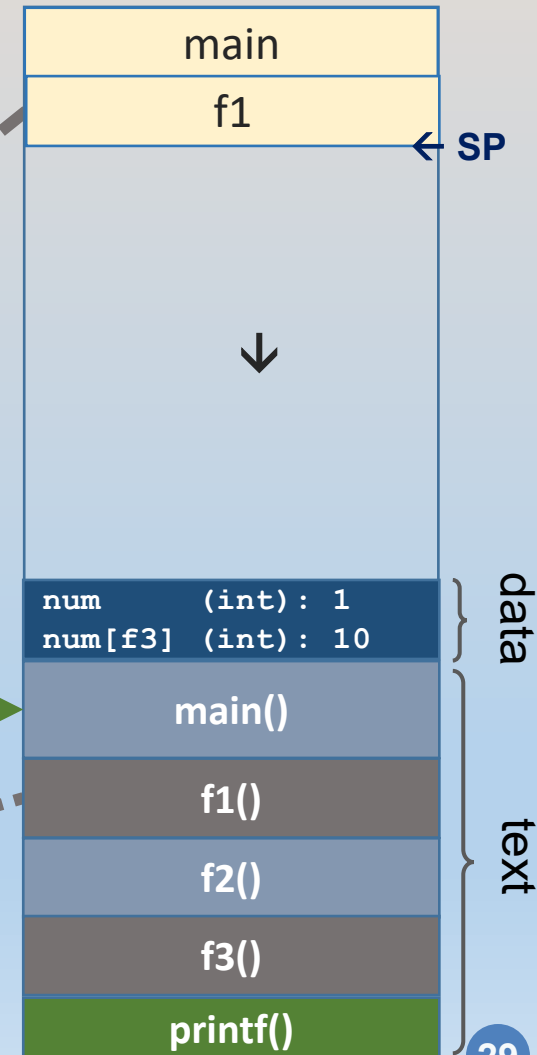
Program execution example #1

- Function f1() is invoked and its stack frame is pushed
- PC moves to the appropriate code block
- Return address (in main) is saved in stack frame
- Variable `num` accessed in `num++` operation is local var (in stack frame)
- PC returns to location in main

return address:	(in main)
parameters:	- none -
local vars:	num (int): 101

```

f1:
    num++;
PC → return num;
    
```



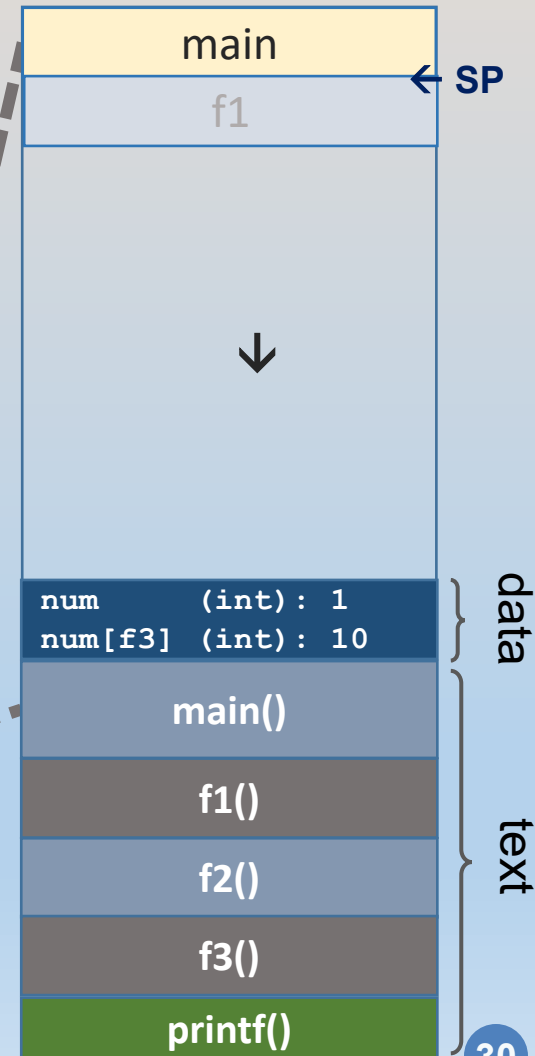
Program execution example #1

- When f1() returns, PC moves to saved return address and f1's stack frame is popped
- Stack frame is popped by moving up the stack pointer (SP). Content is not erased!
- Now printf() is called

```

main:
    i = 0;
PC→ f1(); printf(. . .);
    f2(); printf(. . .);
    f3(); printf(. . .);
    i = 1;
    f1(); printf(. . .);
    f2(); printf(. . .);
    f3(); printf(. . .);
    . . .
    
```

return address:	- system -
parameters:	- none -
local vars:	i(int): 0



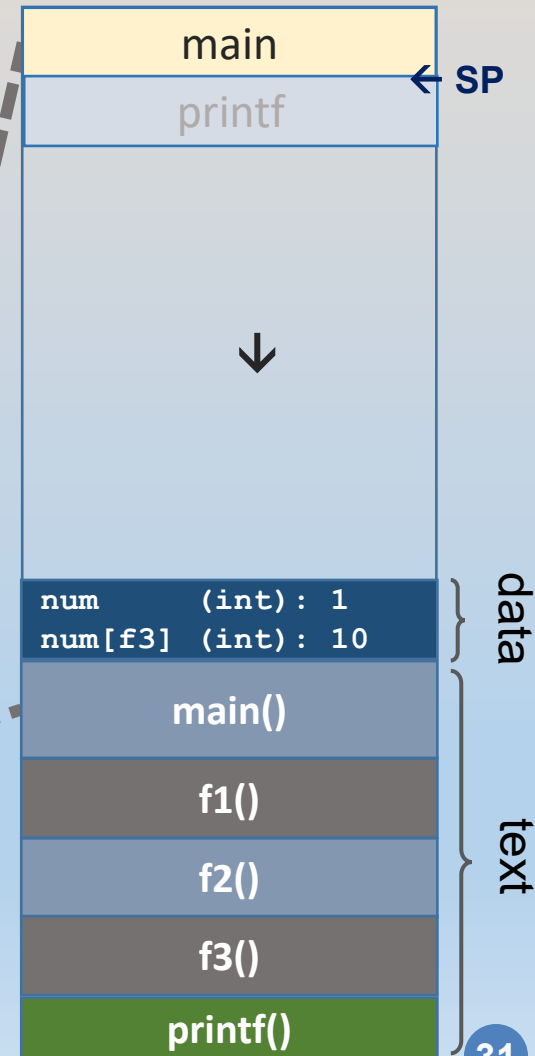
Program execution example #1

- Now f2() is invoked ...

```

main:
    i = 0;
    f1(); printf(. . .);
PC→ f2(); printf(. . .);
    f3(); printf(. . .);
    i = 1;
    f1(); printf(. . .);
    f2(); printf(. . .);
    f3(); printf(. . .);
    . . .
    
```

return address:	- system -
parameters:	- none -
local vars:	i(int): 0



Program execution example #1

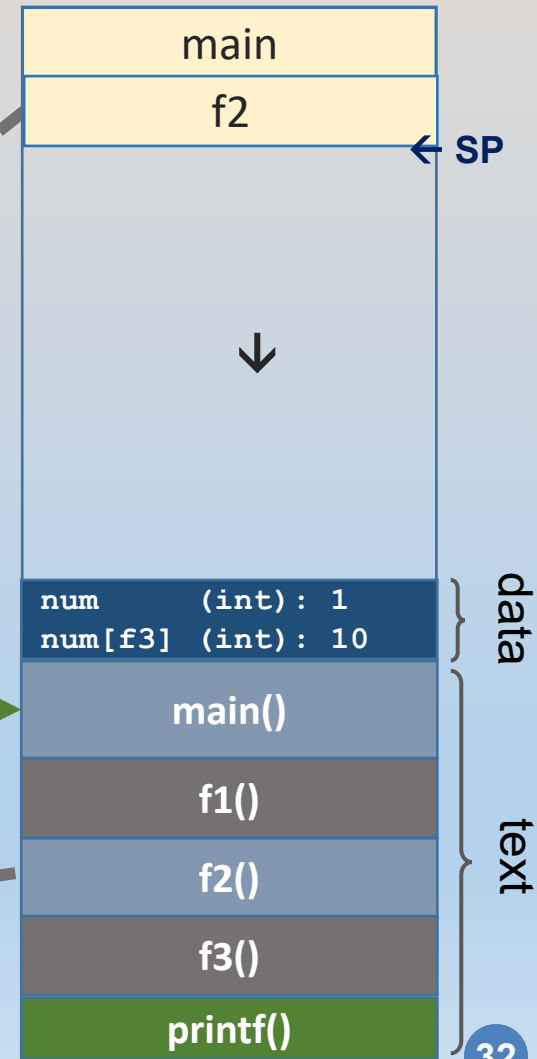
- Now f2() is invoked and its stack frame is pushed

return address:	(in main)
parameters:	- none -
local vars:	- none -

- PC moves to the appropriate code block
- Return address (in main) is saved in stack frame
- Variable `num` accessed in `num++` operation is global (extern) var (in data segment)

```

f2:
PC→ num++;
    return num;
    
```



Program execution example #1

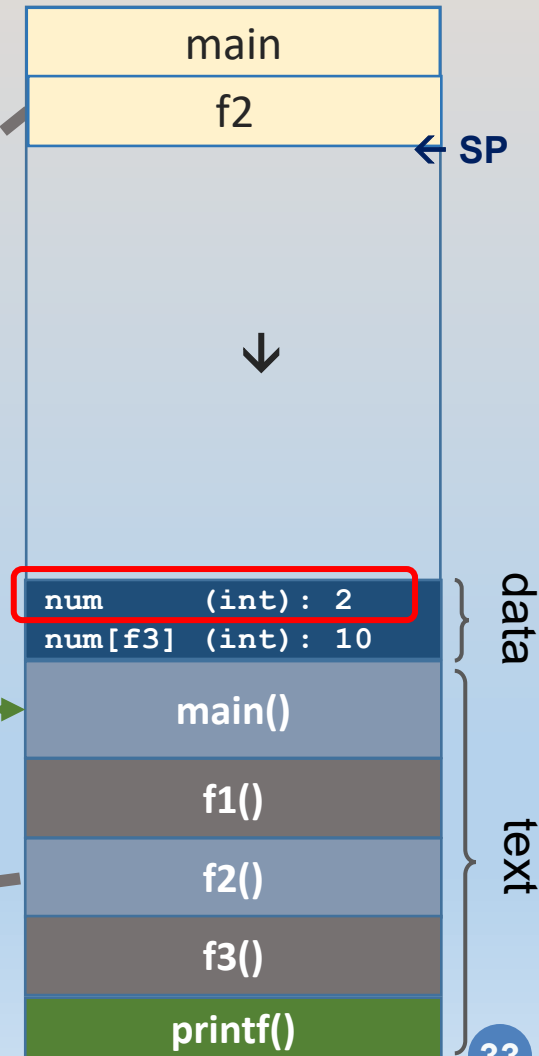
- Now f2() is invoked and its stack frame is pushed

return address:	(in main)
parameters:	- none -
local vars:	- none -

- PC moves to the appropriate code block
- Return address (in main) is saved in stack frame
- Variable `num` accessed in `num++` operation is global (extern) var (in data segment)
- PC returns to location in main

```

f2:
    num++;
PC→ return num;
    
```



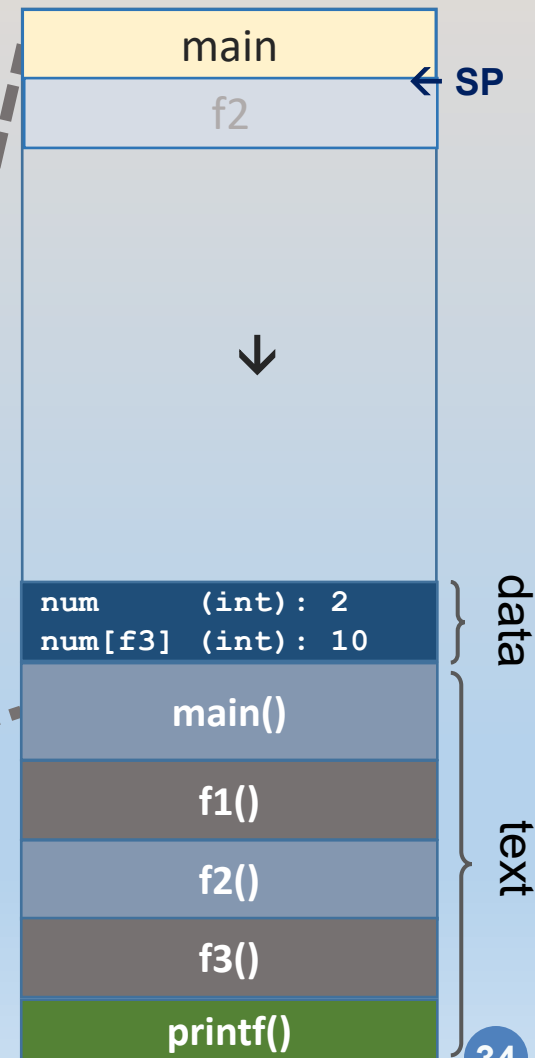
Program execution example #1

- When f2() returns, PC moves to saved return address and f2's stack frame is popped
- Stack frame is popped by moving up the stack pointer (SP). Content is not erased!
- Now printf() is called

```

main:
    i = 0;
    f1(); printf(. . .);
PC→ f2(); printf(. . .);
    f3(); printf(. . .);
    i = 1;
    f1(); printf(. . .);
    f2(); printf(. . .);
    f3(); printf(. . .);
    . . .
    
```

return address:	- system -
parameters:	- none -
local vars:	i(int): 0



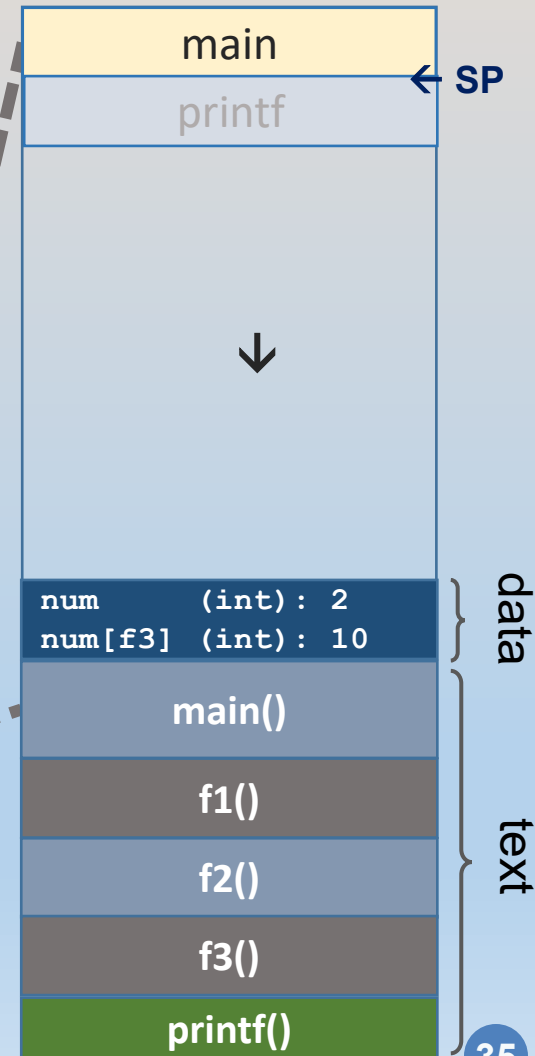
Program execution example #1

- Now f3() is invoked ...

```

main:
    i = 0;
    f1(); printf(. . .);
    f2(); printf(. . .);
    PC→ f3(); printf(. . .);
    i = 1;
    f1(); printf(. . .);
    f2(); printf(. . .);
    f3(); printf(. . .);
    . . .
    
```

return address:	- system -
parameters:	- none -
local vars:	i(int): 0



Program execution example #1

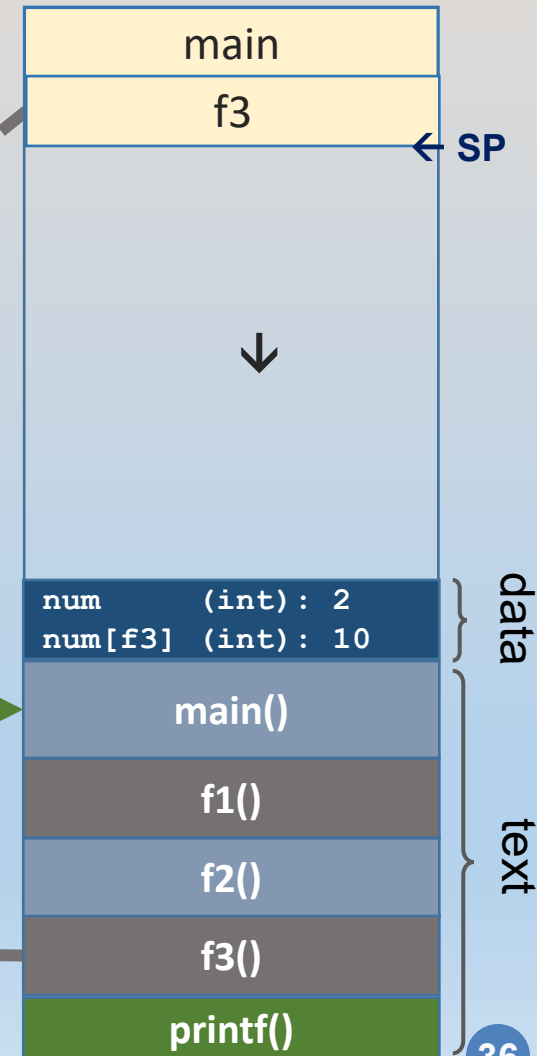
- Now f3() is invoked and its stack frame is pushed

return address:	(in main)
parameters:	- none -
local vars:	- none -

- PC moves to the appropriate code block
- Return address (in main) is saved in stack frame
- Variable `num` accessed in `num++` operation is local static var (in data segment)

```

f3:
PC→ num++;
    return num;
    
```



Program execution example #1

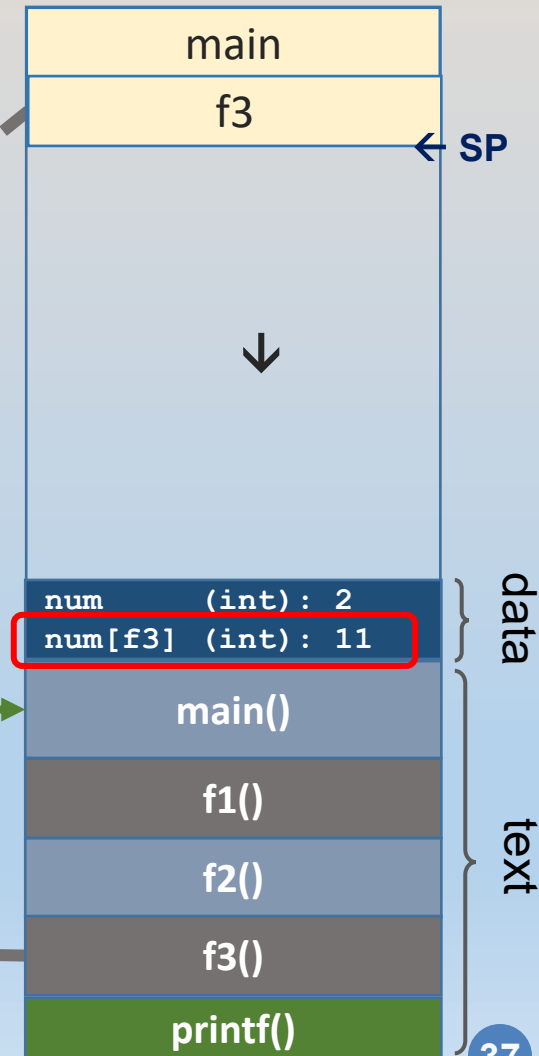
- Now f3() is invoked and its stack frame is pushed

return address:	(in main)
parameters:	- none -
local vars:	- none -

- PC moves to the appropriate code block
- Return address (in main) is saved in stack frame
- Variable `num` accessed in `num++` operation is local static var (in data segment)
- PC returns to location in main

```

f3:
    num++;
PC→ return num;
    
```



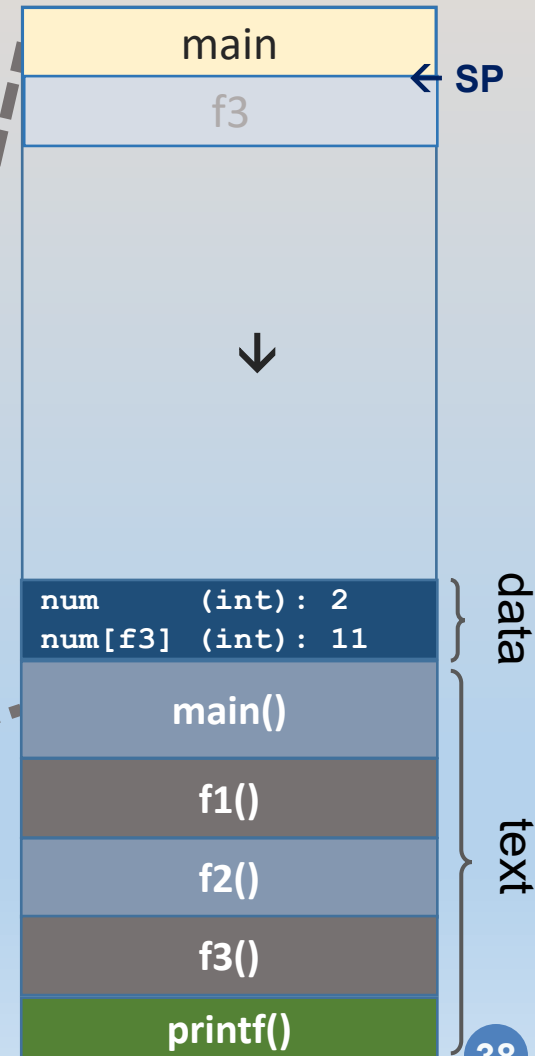
Program execution example #1

- When f3() returns, PC moves to saved return address and f3's stack frame is popped
- Stack frame is popped by moving up the stack pointer (SP). Content is not erased!
- Now printf() is called . . .

```

main:
    i = 0;
    f1(); printf(. . .);
    f2(); printf(. . .);
    PC→ f3(); printf(. . .);
    i = 1;
    f1(); printf(. . .);
    f2(); printf(. . .);
    f3(); printf(. . .);
    . . .
    
```

return address:	- system -
parameters:	- none -
local vars:	i(int): 0



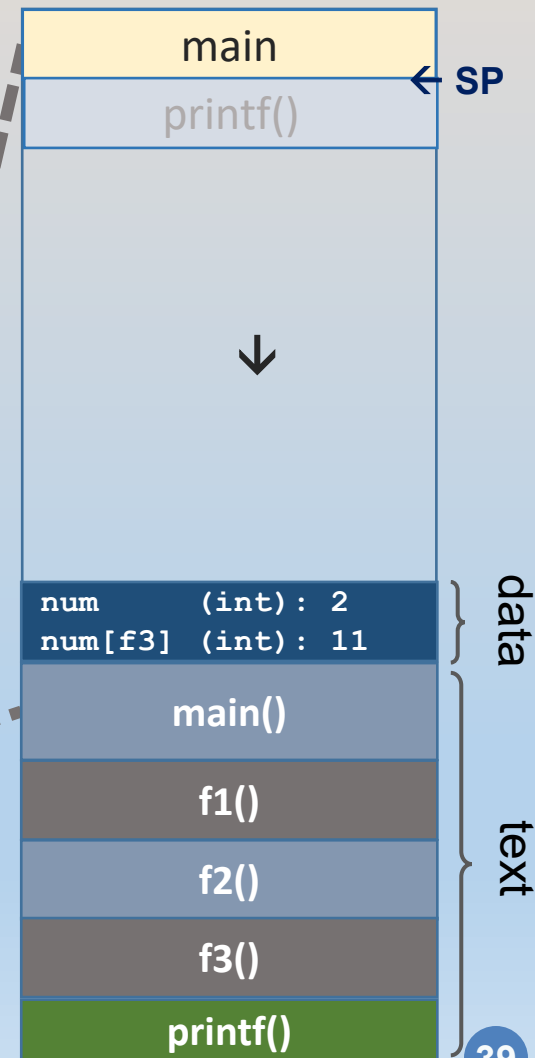
Program execution example #1

- And so on...
- When f1() is re-invoked, we get the same output because it accesses a re-initialized local var
- The vars that f2() and f3() access are in data segment, so their values are kept from one invocation to the next

```

main:
    i = 0;
    f1(); printf(. . .);
    f2(); printf(. . .);
    f3(); printf(. . .);
    i = 1;
PC→ f1(); printf(. . .);
    f2(); printf(. . .);
    f3(); printf(. . .);
    . . .
    
```

return address:	- system -
parameters:	- none -
local vars:	i(int): 1



Variable scope + life span

Local (auto) variables:

- Allocated **on call stack**
- Scope: accessible when function is invoked, released when it returns
- Life span: alive as long as its function is active (on stack)

Global variables + Local static (extern) variables:

- Allocated **in data segment**
- Scope: global accessible from that point forward/static accessible in that code block
- Life span: alive throughout the run time of the program

Local vars are **not initialized** (unless explicitly assigned a value)

Static and global vars are **initialized to 0** (unless explicitly assigned a different value)

Program execution example #2

code in `/share/classes/class07/power.c`

```
#include <stdio.h>

/** function declarations */
double power(double, int);    // computes power (x^n)
double square(double);        // computes square (x^2)

/** global variables */
int function_counter = 0;     // computes number of function invocations

/** main */
int main() {
    double x=0.5, res;
    int n=5;

    res = power(x,n);
    printf("Computed %lf^%d = %lf using %d function calls\n",x,n,res, function_counter );

    return 0;
}
/** end of main() */
```

Program execution example #2

```

. . .
/*****
function square
. . . Some doc here (see code file) . . .
*****/
double square(double x) {
    function_counter++;
    return x*x;
}

/*****
function power
. . . Some doc here (see code file) . . .
*****/
double power(double x, int n) {
    double y=1.0;
    function_counter++;

    if(n < 0) {
        y = 1 / power(x,-n);
    } else if (n>0) {
        y = square(power(x,n/2));    // <-- y = (x^(n/2))^2
        if(n%2 == 1) y = y*x;
    }
    return y;
}

```

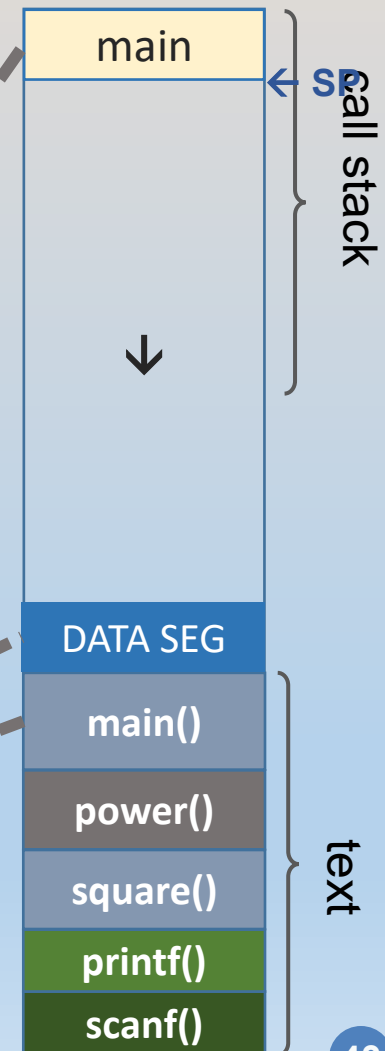
Program execution example #2

- Program loads to text segment of RAM
- Global variable is allocated and initialized
- Stack is initialized with stack frame for main()
- Uninitialized local variables may have any value (garbage)

return address:	- system -
parameters:	- none -
local vars:	x (double): 0.5
	res (double): ?
	n (int): 5

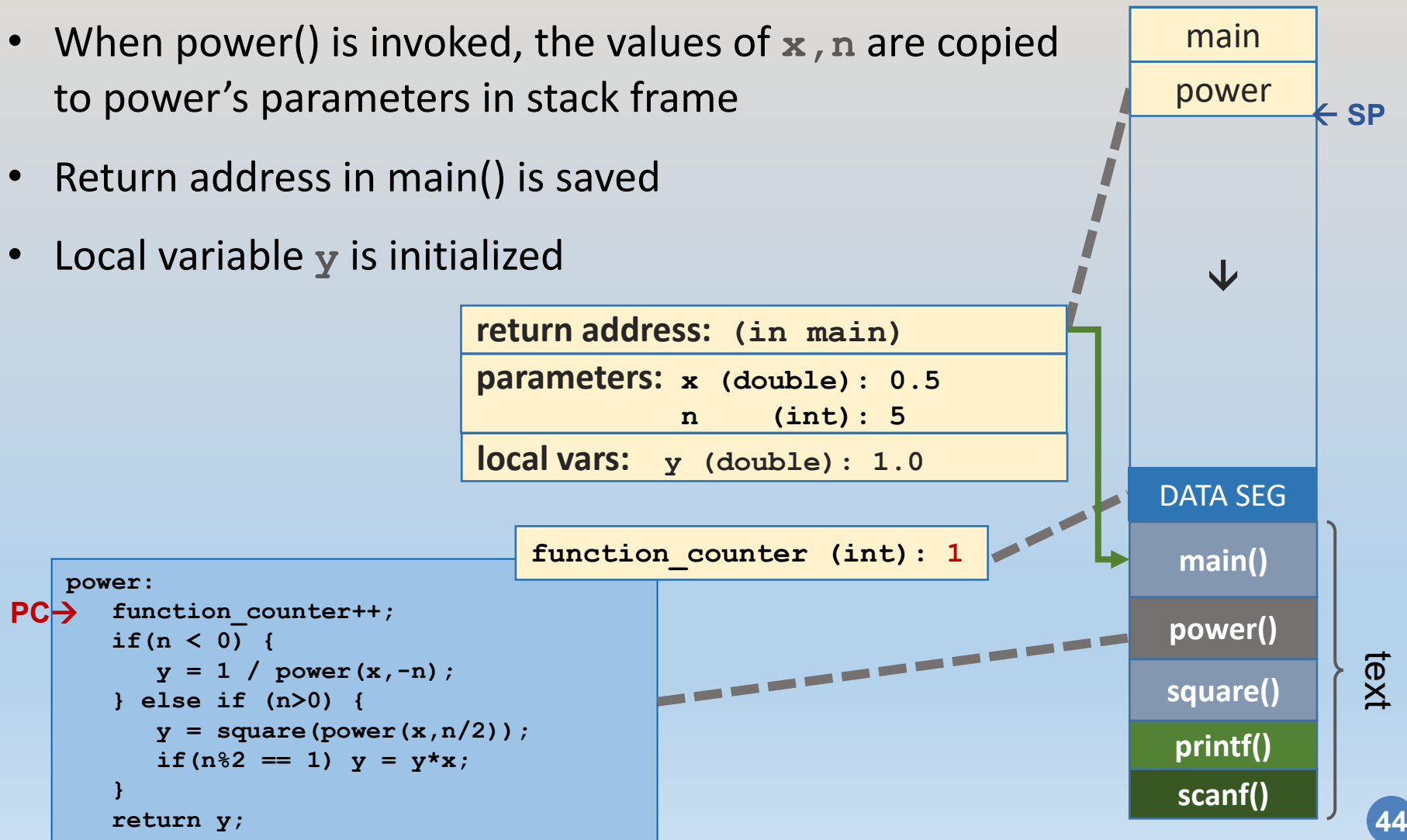
function_counter (int): 0

PC →
 main:
 res = power(x,n);
 printf(. . .);
 return 0;



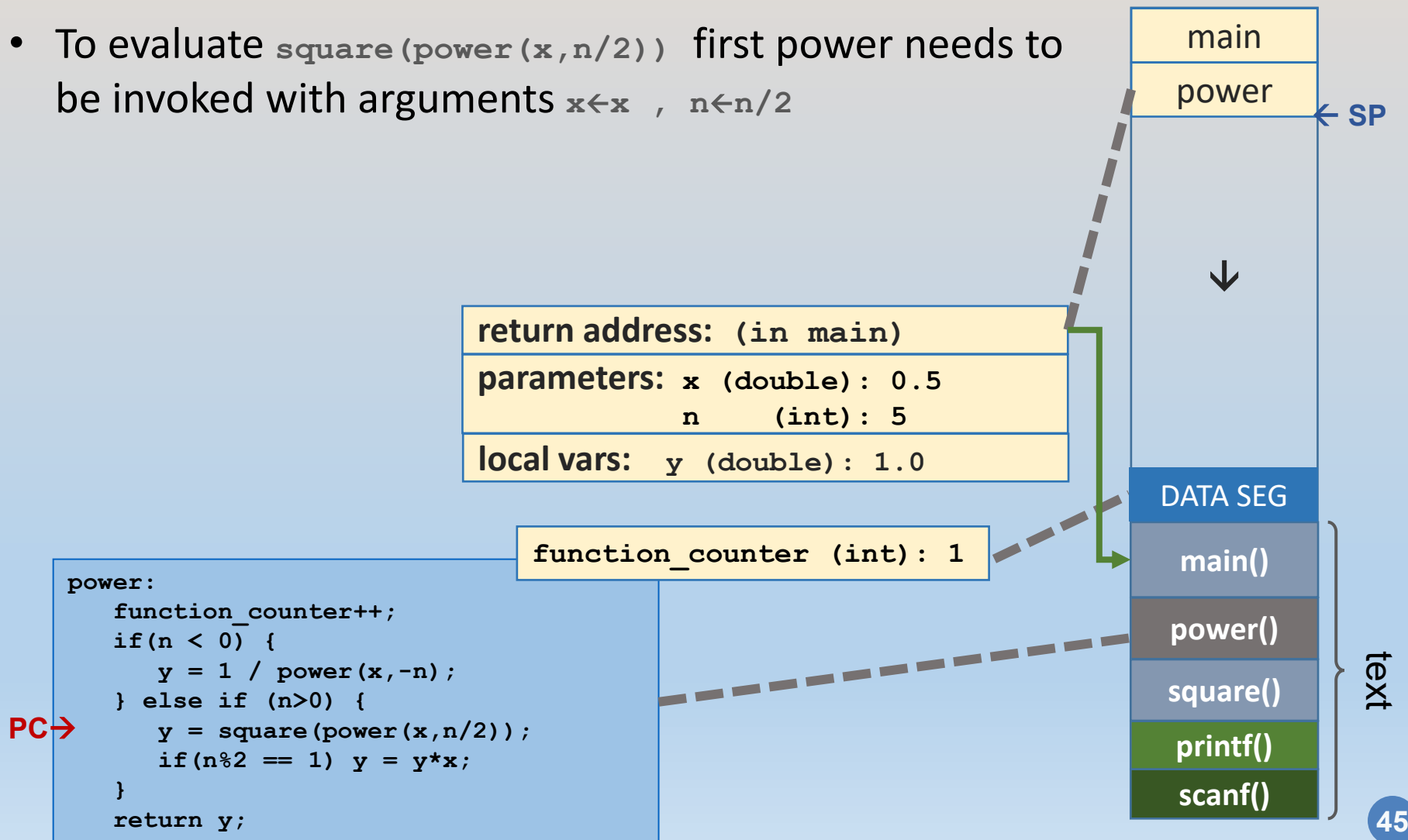
Program execution example #2

- When `power()` is invoked, the values of `x`, `n` are copied to `power`'s parameters in stack frame
- Return address in `main()` is saved
- Local variable `y` is initialized



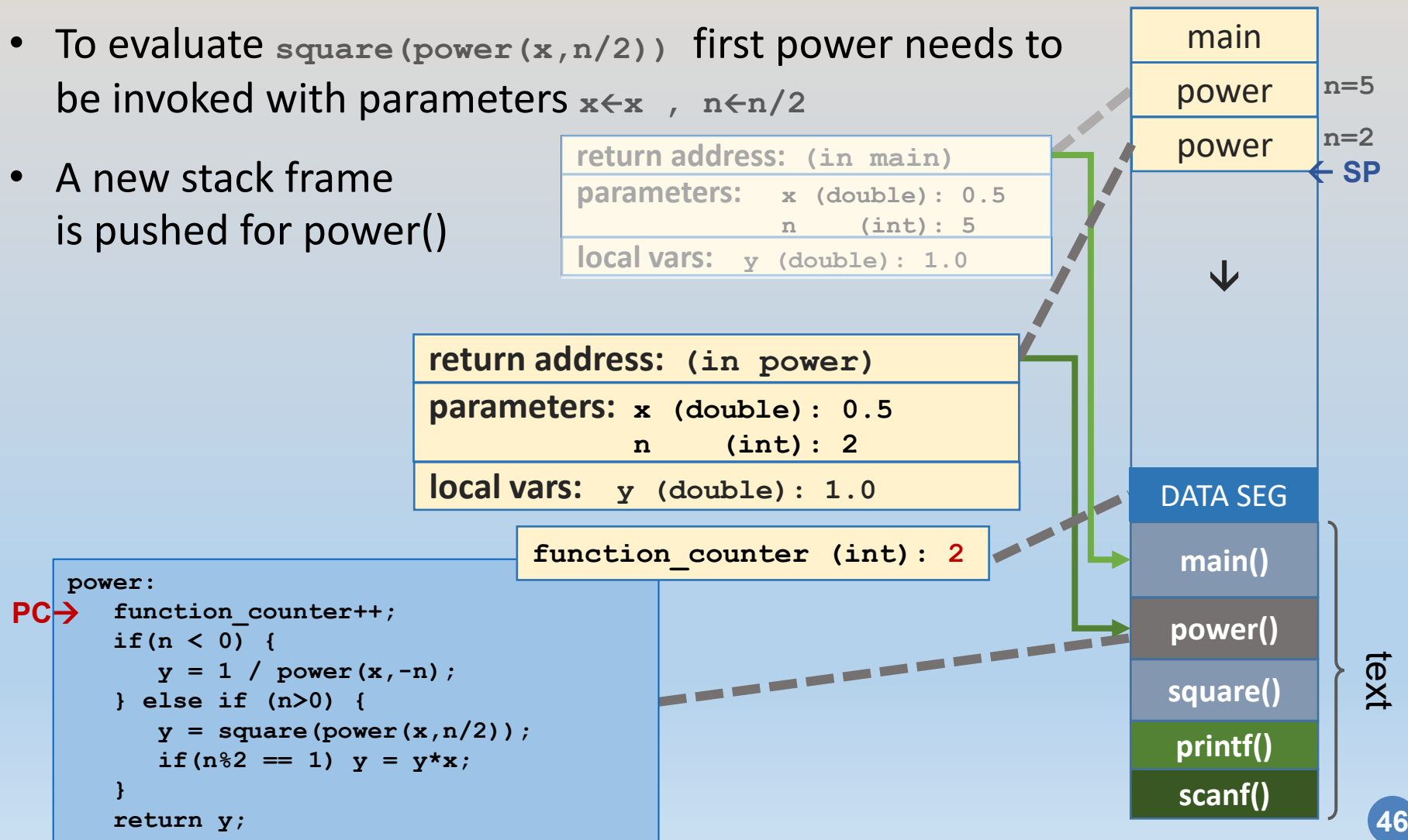
Program execution example #2

- To evaluate `square(power(x, n/2))` first power needs to be invoked with arguments $x \leftarrow x$, $n \leftarrow n/2$



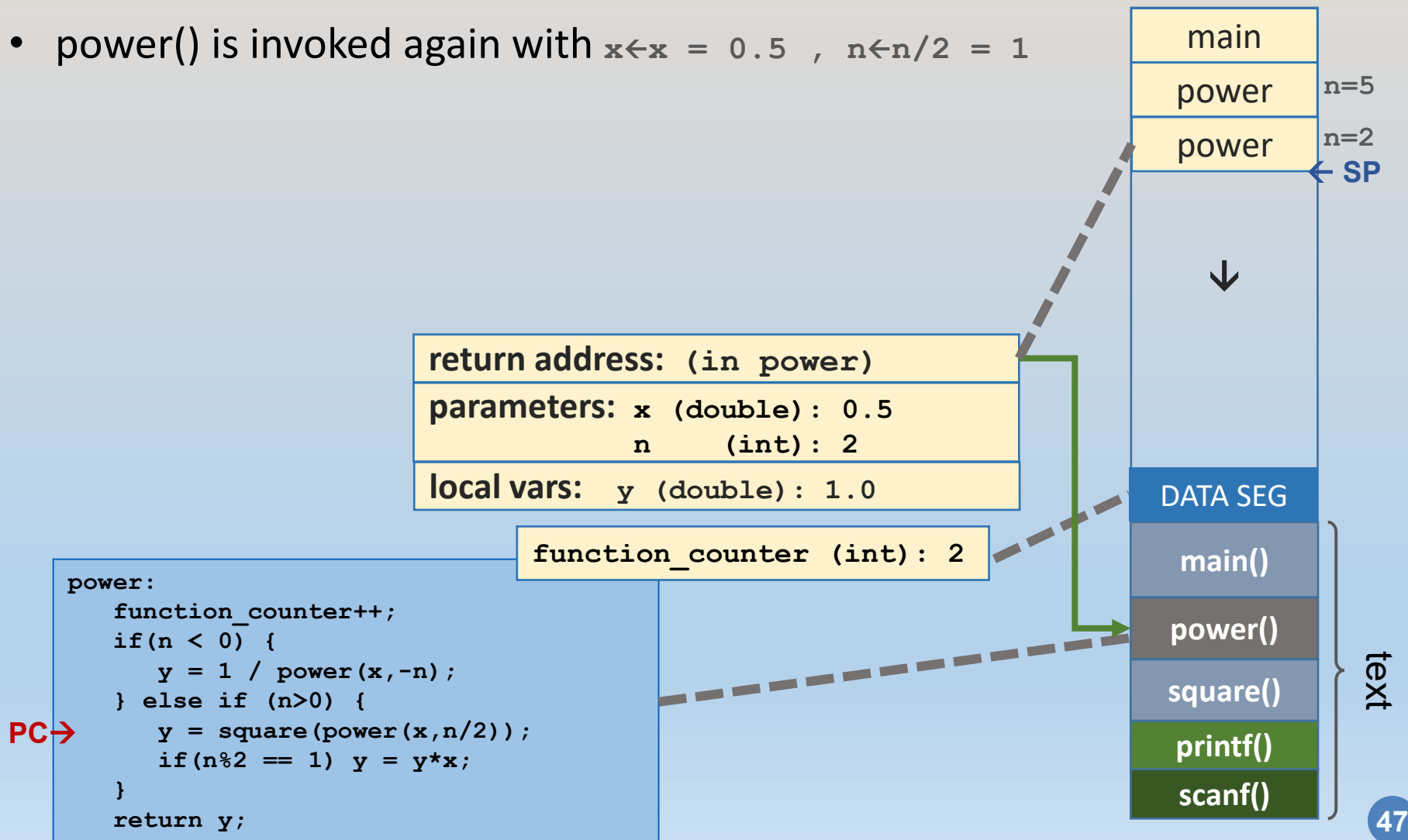
Program execution example #2

- To evaluate `square(power(x, n/2))` first power needs to be invoked with parameters $x \leftarrow x$, $n \leftarrow n/2$
- A new stack frame is pushed for `power()`



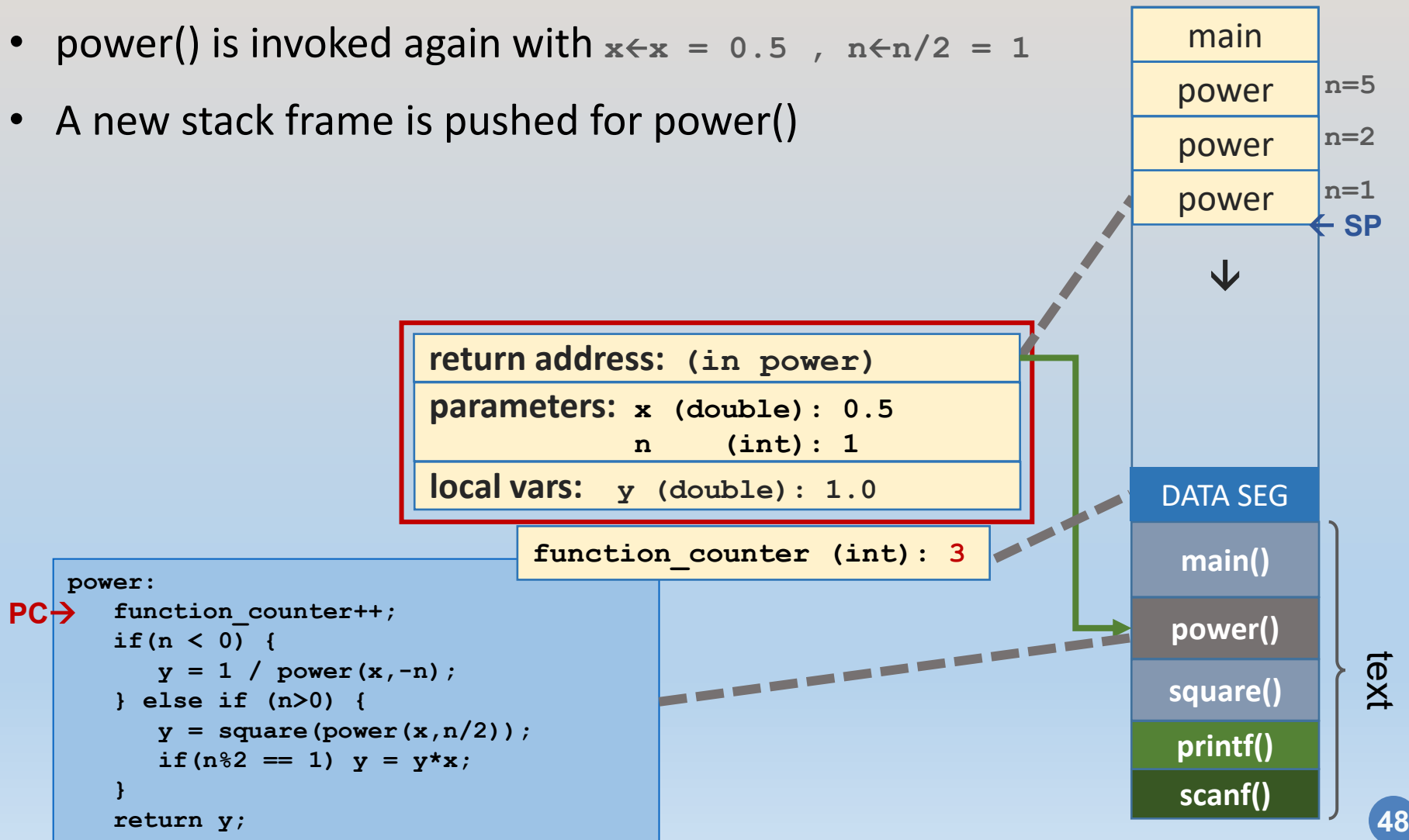
Program execution example #2

- power() is invoked again with $x \leftarrow x = 0.5$, $n \leftarrow n/2 = 1$



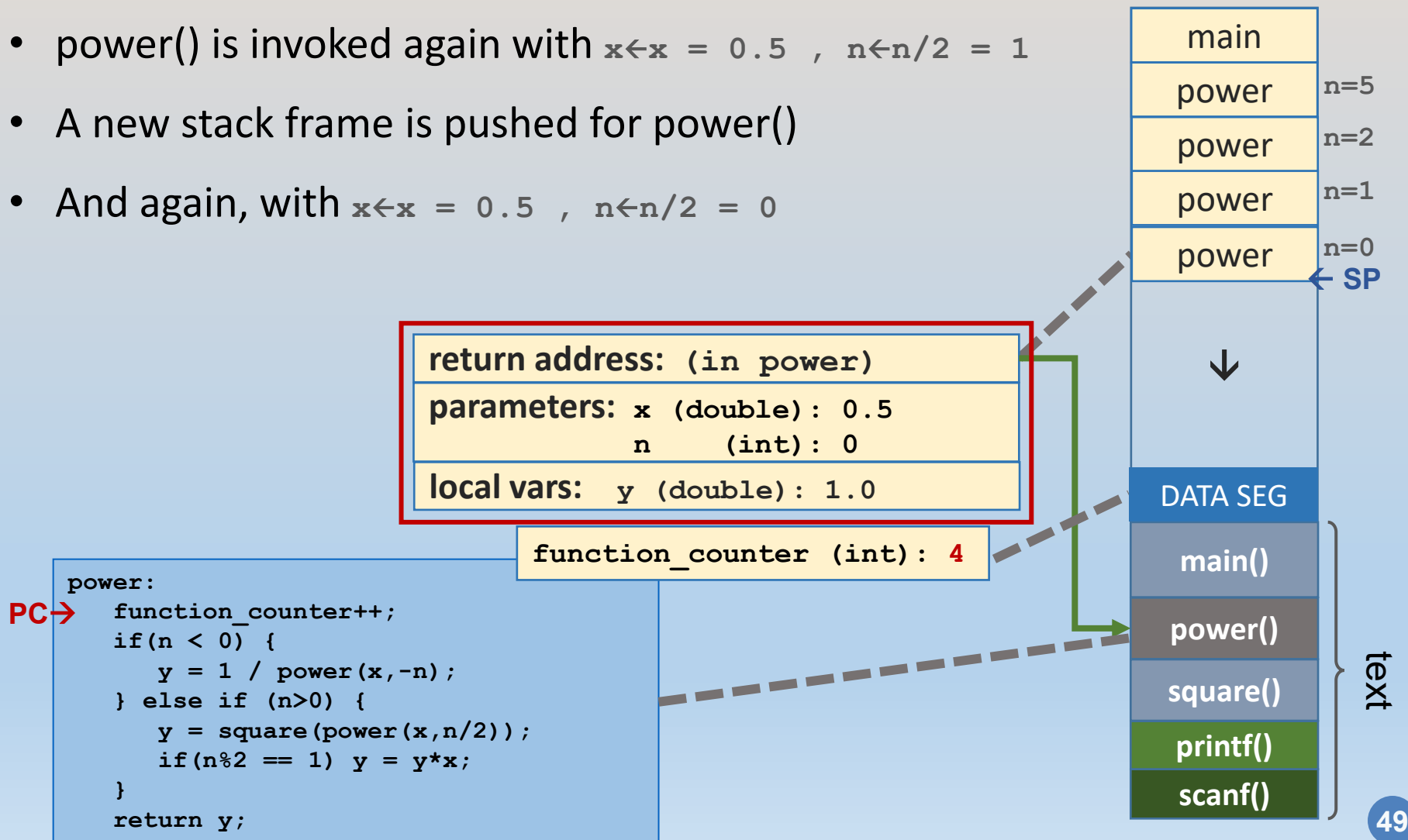
Program execution example #2

- `power()` is invoked again with $x \leftarrow x = 0.5$, $n \leftarrow n/2 = 1$
- A new stack frame is pushed for `power()`



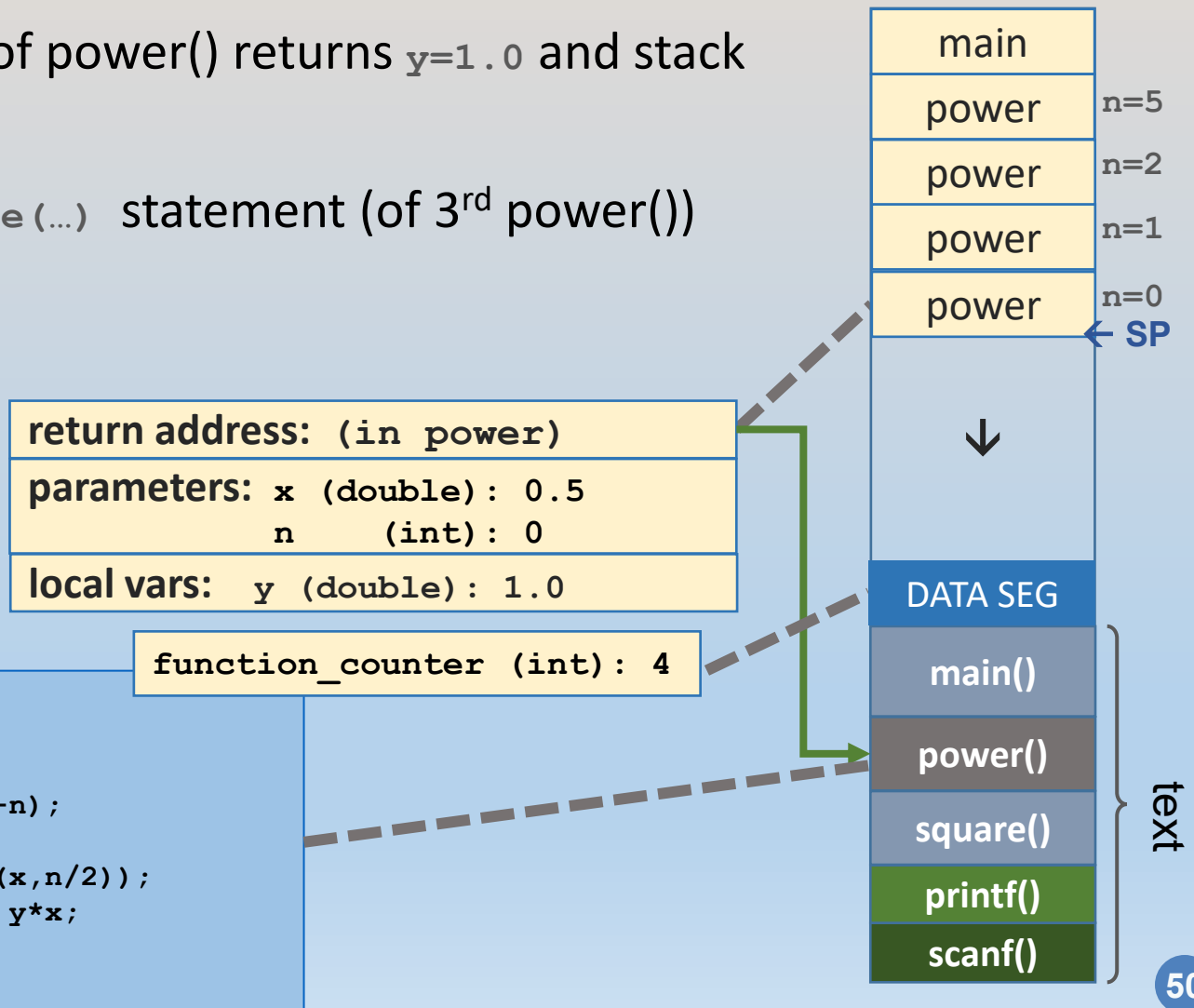
Program execution example #2

- `power()` is invoked again with $x \leftarrow x = 0.5$, $n \leftarrow n/2 = 1$
- A new stack frame is pushed for `power()`
- And again, with $x \leftarrow x = 0.5$, $n \leftarrow n/2 = 0$



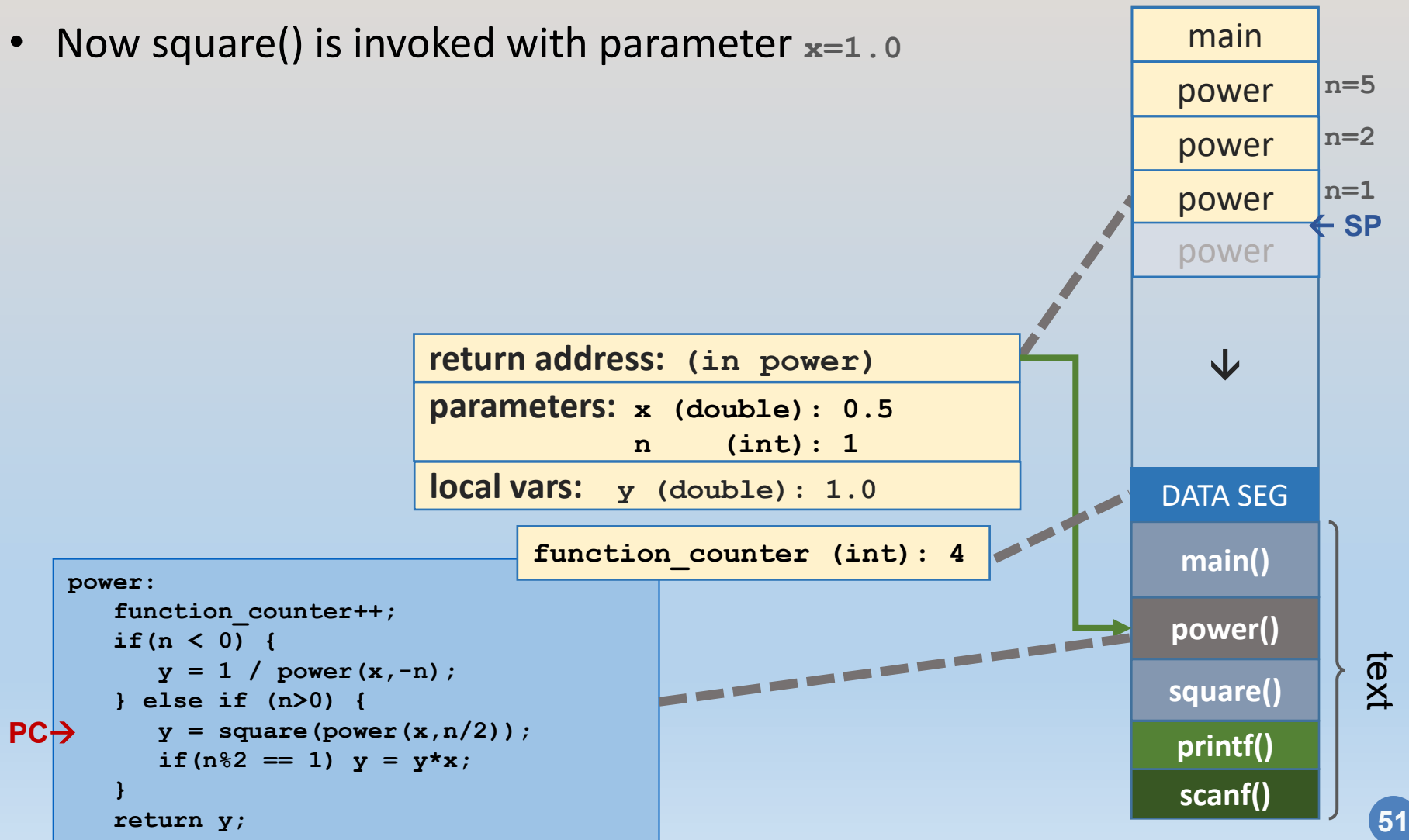
Program execution example #2

- The 4th invocation of power() returns $y=1.0$ and stack frame is popped
- PC returns to `square(...)` statement (of 3rd power())



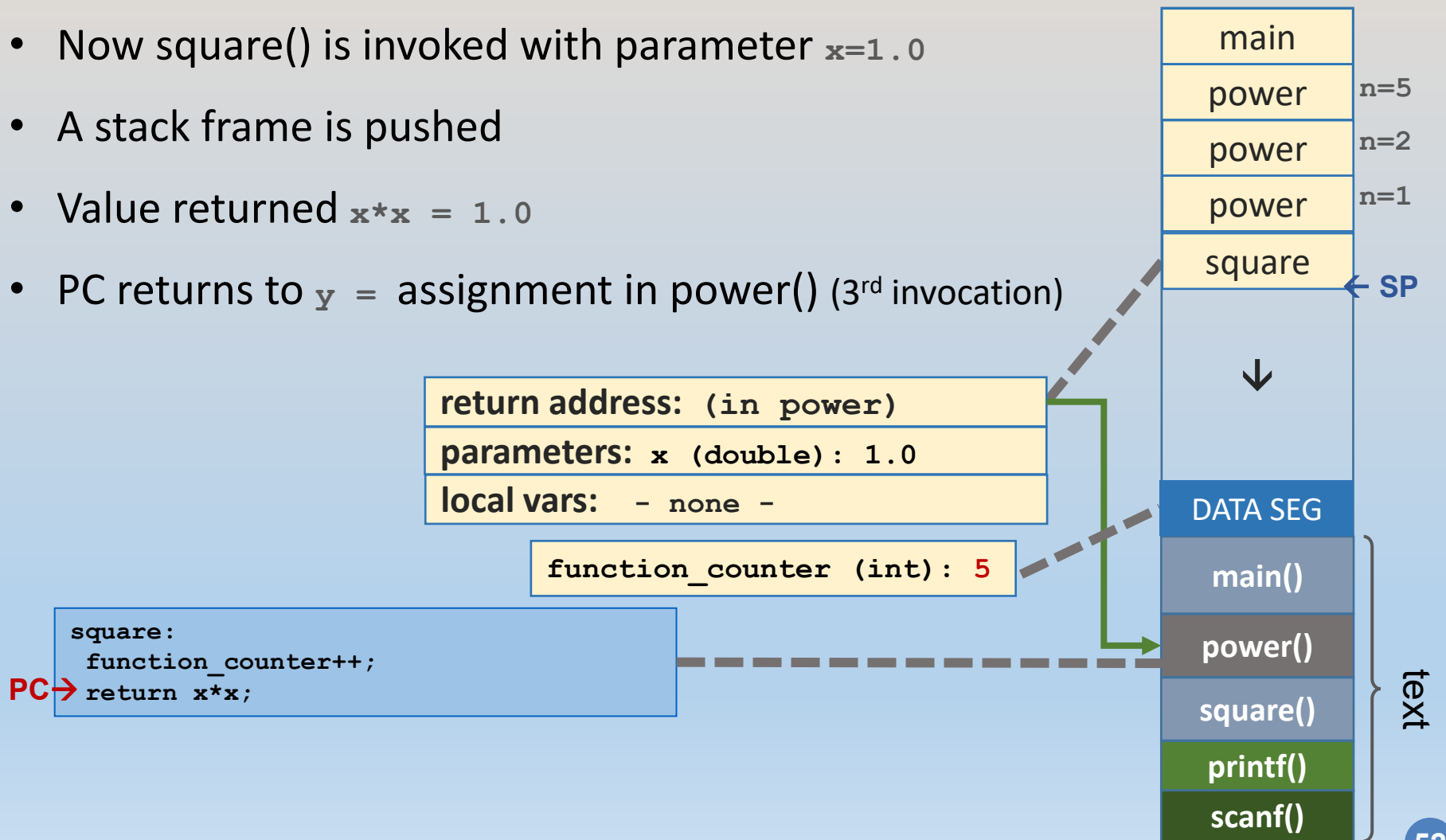
Program execution example #2

- Now square() is invoked with parameter `x=1.0`



Program execution example #2

- Now square() is invoked with parameter `x=1.0`
- A stack frame is pushed
- Value returned `x*x = 1.0`
- PC returns to `y = assignment in power()` (3rd invocation)



Program execution example #2

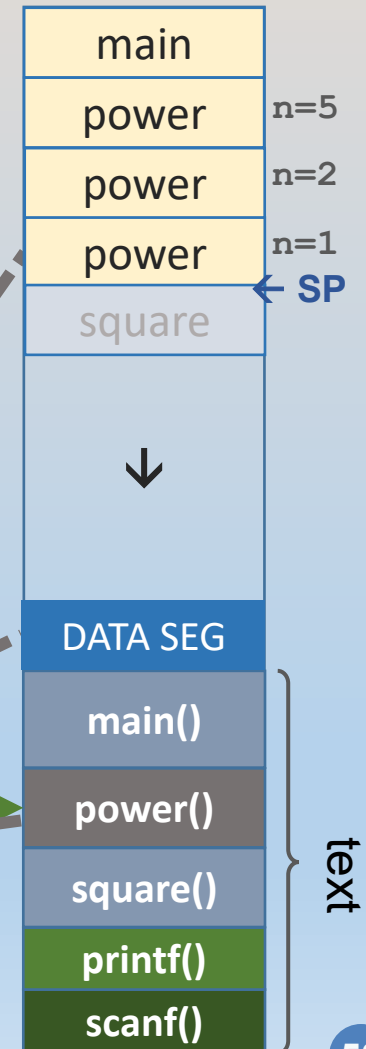
- Local var `y` gets assigned return value of `square()` $y \leftarrow 1.0$
- Because $n \% 2 == 1$, $y \leftarrow y * x = 1.0 * 0.5 = 0.5$
- The 3rd invocation of `power()` returns $y=0.5$ and stack frame is popped. PC returns to `square(...)` statement (of 2nd `power()`)

return address: (in power)
 parameters: `x` (double): 0.5
 `n` (int): 1
 local vars: `y` (double): 0.5

function_counter (int): 5

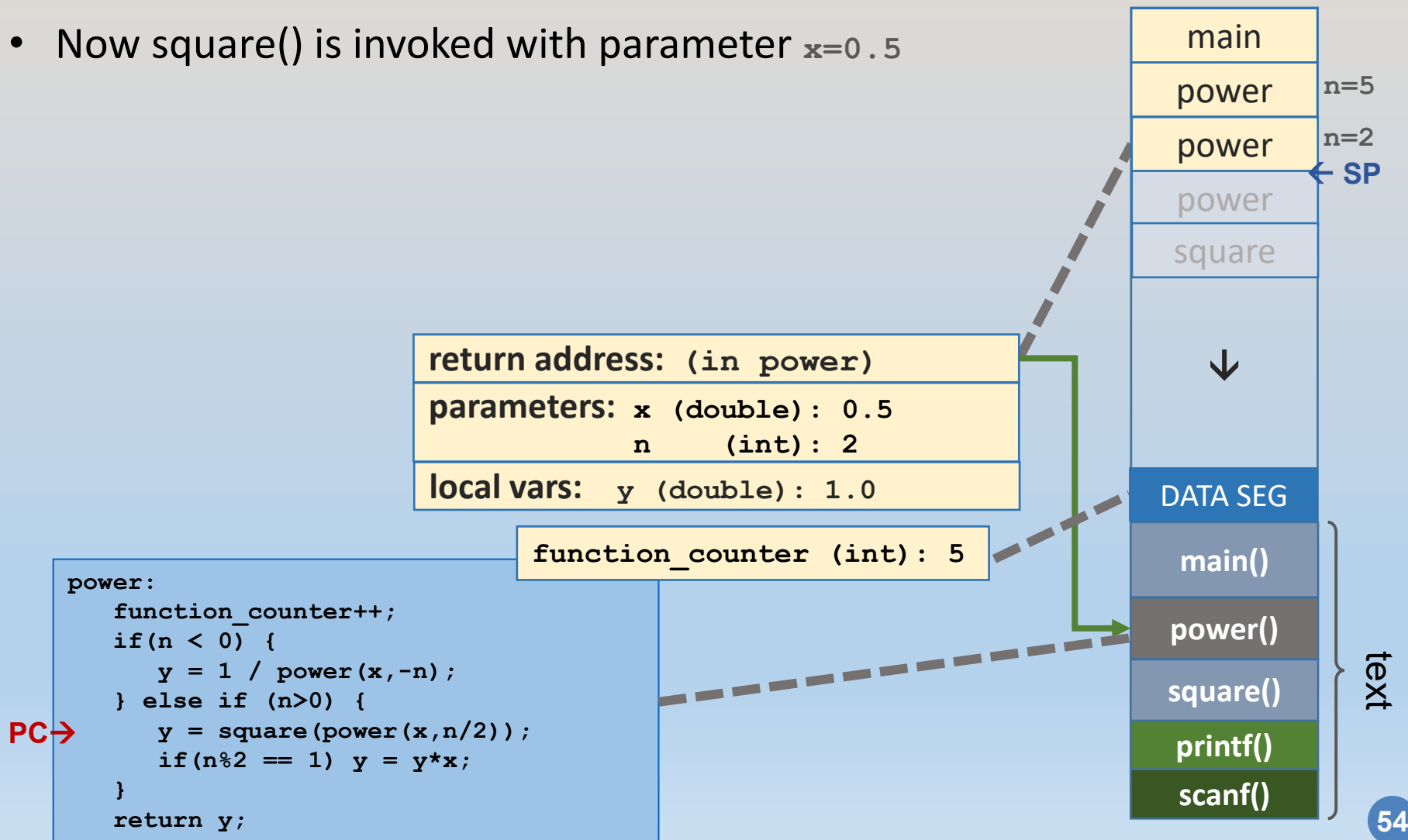
PC →

```
power:
    function_counter++;
    if(n < 0) {
        y = 1 / power(x, -n);
    } else if (n > 0) {
        y = square(power(x, n/2));
        if(n%2 == 1) y = y*x;
    }
    return y;
```



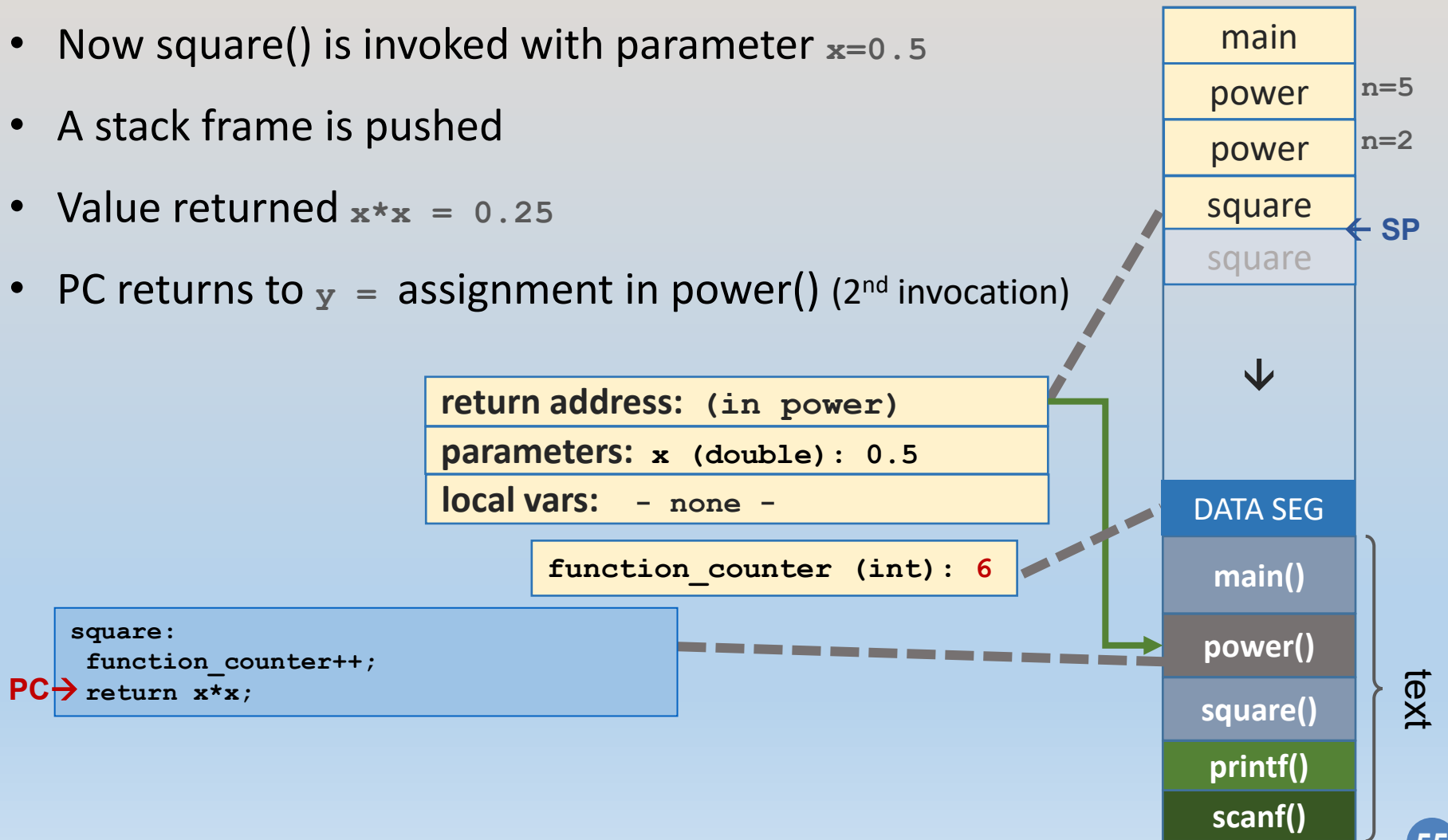
Program execution example #2

- Now square() is invoked with parameter $x=0.5$



Program execution example #2

- Now square() is invoked with parameter $x=0.5$
- A stack frame is pushed
- Value returned $x*x = 0.25$
- PC returns to y = assignment in power() (2nd invocation)



Program execution example #2

- And so forth, and so forth . . .
- Current stack frame of power() is popped
- Another stack frame of square() is loaded (`function_counter` incremented to 7)
- Stack frames of power() and square() are popped

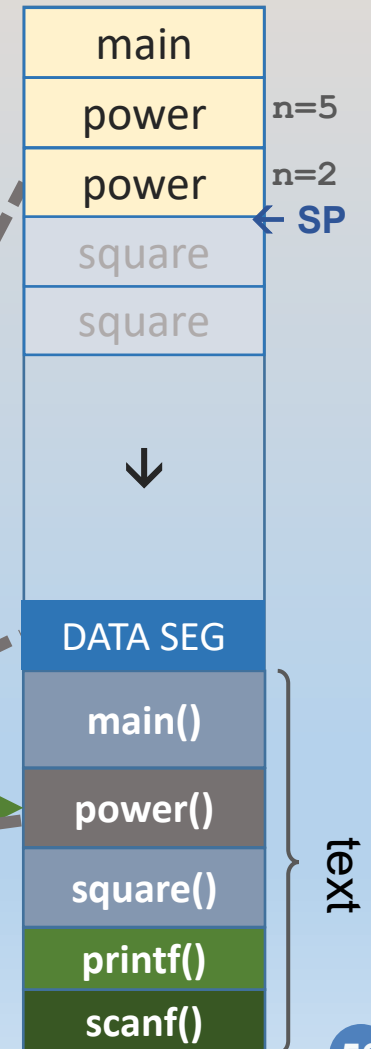
return address: (in power)
 parameters: `x (double): 0.5`
 `n (int): 2`
 local vars: `y (double): 0.25`

`function_counter (int): 6`

PC →

```

power:
  function_counter++;
  if(n < 0) {
    y = 1 / power(x, -n);
  } else if (n > 0) {
    y = square(power(x, n/2));
    if(n%2 == 1) y = y*x;
  }
  return y;
  
```



Program execution example #2

- When PC returns to main, return value is assigned to `res`
- result is printed using `printf()` and program exits (`return 0`)

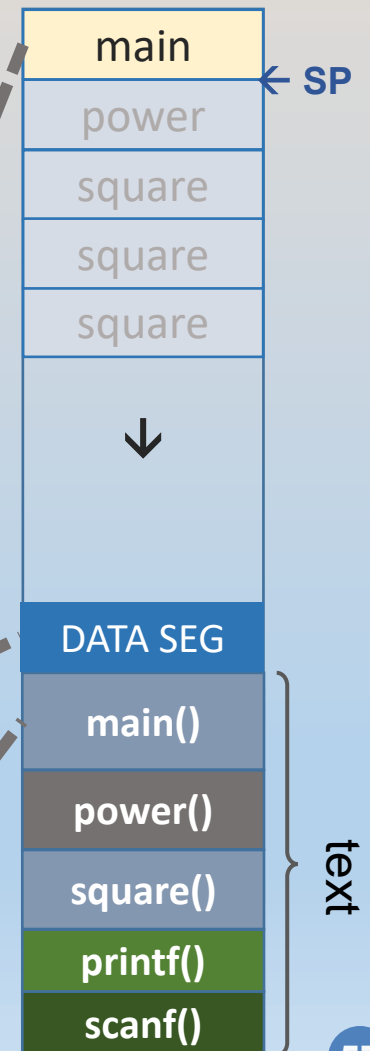
return address:	- system -
parameters:	- none -
local vars:	x (double): 0.5 res (double): 0.03125 n (int): 5

`function_counter (int): 7`

```

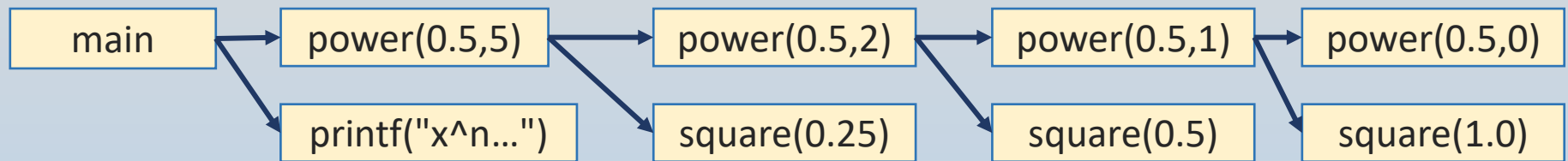
PC → main:
    res = power(x,n);
    printf("Computed %lf^%d = %lf using %d function calls\n",
        x,n,res, function_counter );

    return 0;
    
```



Function call tree

- Function invocations of program execution can be described using the call tree



- Program execution is like a Depth First Search (DFS) traversal of this tree
- At every point in the execution of a program, the call stack contains stack frames for all functions in the path from root (= main) to current node
 - Moving forward on edge = invoking a function and pushing stack frame
 - Moving back on edge = returning from function and popping stack frame

Online poll

If you do not assign a local (automatic) variable a value, then

- Its value is automatically initialized to 0
- The compiler will print an error message
- The program will halt with an error message
- It is assigned some arbitrary value, which is the same every time the function is called
- It is assigned some arbitrary value, which can be different every time the function is called

If you do not assign a (local) static variable a value, then

- Its value is automatically initialized to 0
- The compiler will print an error message
- The program will halt with an error message
- It is assigned some arbitrary value, which is the same every time the function is called
- It is assigned some arbitrary value, which can be different every time the function is called

Online poll – solutions

If you do not assign a local (automatic) variable a value, then

- Its value is automatically initialized to 0
- The compiler will print an error message
- The program will halt with an error message
- It is assigned some arbitrary value, which is the same every time the function is called
- It is assigned some arbitrary value, which can be different every time the function is called

← correct answer

If you do not assign a (local) static variable a value, then

- Its value is automatically initialized to 0
- The compiler will print an error message
- The program will halt with an error message
- It is assigned some arbitrary value, which is the same every time the function is called
- It is assigned some arbitrary value, which can be different every time the function is called

← correct answer

Online poll

If you do not assign a global (extern) variable a value, then

- Its value is automatically initialized to 0
- The compiler will print an error message
- The program will halt with an error message
- It is assigned some arbitrary value

The initial value of a static (local) variable is set

- The first time the function is called
- Every time the function is called
- When the program is compiled
- When the program starts running

Which of the following is true?

- Every function always has exactly one stack frame on the stack
- Every function always has at least one stack frame on the stack
- Every function always has at most one stack frame on the stack
- None of the above

Online poll – solutions

If you do not assign a global (extern) variable a value, then

- Its value is automatically initialized to 0 ← **correct answer**
- The compiler will print an error message
- The program will halt with an error message
- It is assigned some arbitrary value

The initial value of a static (local) variable is set

- The first time the function is called
- Every time the function is called
- When the program is compiled ← **correct answer**
- When the program starts running

Which of the following is true?

- Every function always has exactly one stack frame on the stack
- Every function always has at least one stack frame on the stack
- Every function always has at most one stack frame on the stack
- None of the above ← **correct answer**

Lecture overview

- Function declarations and definitions – recap
- Variable scope and storage classes
- Program memory segments
- Examples of program execution
- Introduction to pointers ←

Pointers

Each variable in a program is stored at a particular address

- Local variables – in the stack frame of their function
- Global / static variables – in the data segment

We can obtain this address by using the **referencing operator** - &

Pointers

Each variable in a program is stored at a particular address

- Address is typically represented as a hexadecimal integer
e.g., `0xbfffffff924` (8 bytes on our Linux server)
- If `v` is of type `type`, then `&v` is of type `type*`
 - If `v` is of type `int`, then `&v` is of type `int*`
 - If `v` is of type `double`, then `&v` is of type `double*`

Pointers

- Each variable in a program is stored at a particular address
- Address is typically represented as a hexadecimal integer
e.g., `0xbfffffff924` (8 bytes on our Linux server)
 - If `v` is of type `type`, then `&v` is of type `type*`
 - Declaration of pointer variable (to an `int` data type):

```
int *p;  
int* p;  
int*p;  
int * p;
```

all 4 declarations
← of pointer variable
are equivalent

Recommended – otherwise there is
confusion with: `int* p, q;` - what
type is `q`?

Pointers

Each variable in a program is stored at a particular address

- Address is typically represented as a hexadecimal integer
e.g., `0xbfffffff924` (8 bytes on our Linux server)
- If `v` is of type `type`, then `&v` is of type `type*`
- Declaration of pointer variable (to an `int` data type):

```
int *p;
int* p;
int*p;
int * p;
```

all 4 declarations
 ← of pointer variable
 are equivalent

- Pointer assignments:

```
p = &i;
p = NULL;
p = 0;
```

← `i` must be a variable of type `int`

← same effect as `p=NULL` but less
 recommended syntax

Pointers – example

If `p` is of type `int*`, then operator `*` **dereferences** `p`, and `*p` is the `int` value at address `p`

- Print the **address** and **value** of a variable:

```
#include <stdio.h>
int main() {
```

```
    int i = 7;
```

```
    int *ip = &i;
```

```
    printf ("    Value of i: %d\n", *ip);
```

```
    printf ("Location of i: %p\n", ip);
```

```
    return 0;
```

```
}
```

← defining a pointer variable

← dereferencing a pointer

← printing memory address

%p for printing
pointer address

Value of i: 7

Location of i: 0xbfffffff924

address is printed as

← 8-byte hexadecimal code

Pointers – ref and de-ref

The following statement declares and initializes two `int` variables `a`, `b` and declares a pointer to an `int`, `p`

```
int a = 1, b = 2, *p;
```

- We can conceptually represent `p` as an arrow that points to a memory location:

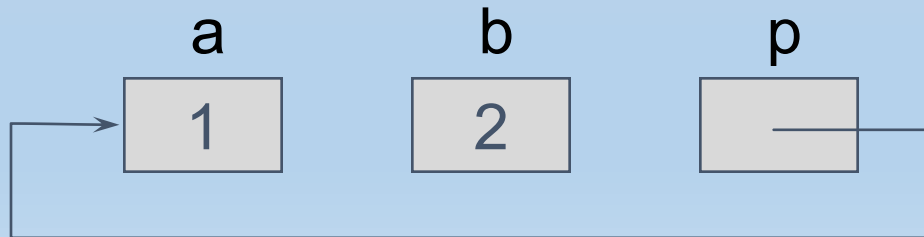


Pointers – ref and de-ref

Now we use **p** to reference **a**

```
int a = 1, b = 2, *p;  
p = &a;
```

- Referencing a variable (using **&**) results in **p** “pointing to” **a**:



Pointers – ref and de-ref

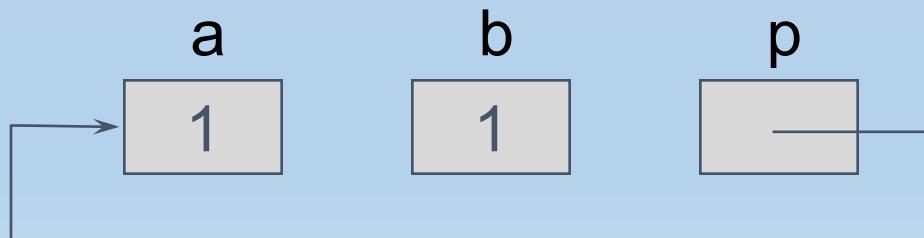
We copy the value of **a** into **b** using the reference **p**

```
int a = 1, b = 2, *p;
p = &a;
b=*p;                /** same as b=a **/
```

Operator precedence :

- Access and Grouping: `()` (function call), `[]`, `.`, `->`
- Unary: `++`, `--`, `!`, `()` (type casting), `&` (address), `*` (dereference)
- Arithmetic: `*` (multiplication), `/`, `%`; and then `+`, `-`
- Bitwise shifts: `<<`, `>>`
- Relational: `>`, `>=`, `<`, `<=`
- Bitwise `&`; and then `^`; and then bitwise `|`
- Logical `&&`; and then logical `||`
- Assignment: `=`, `+=`, `-=`, `*=`, `/=`, etc.

- By dereferencing **p**, we get an alternative way to access **a**

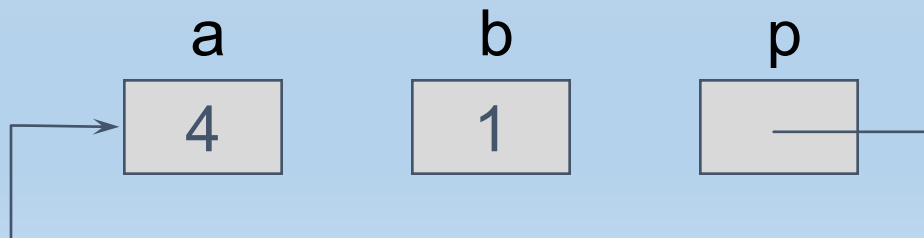


Pointers – ref and de-ref

We copy the value of **a** into **b** using the reference **p**

```
int a = 1, b = 2, *p;  
p = &a;  
b=*p;           /** same as b=a **/  
*p+=3;          /** same as a+=3 **/
```

- By dereferencing **p**, we get an alternative way to access **a**



Referencing / dereferencing

Code example:

```
int      i = 3, j = 5,  
        *p = &i, *q = &j, *r;  
  
if(p == &i) printf ("same address\n");
```

- Pointer types can be compared to each other using all relational operators (==, <, > <=, >=)
- The **addresses** are being compared, and not the content!

Referencing / dereferencing

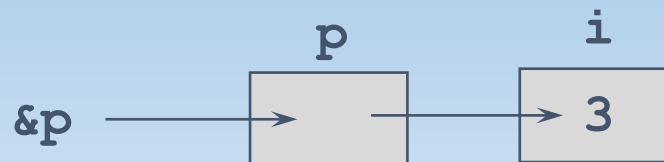
Code example:

```

int      i = 3, j = 5,
        *p = &i, *q = &j, *r;

if(p == &i) printf ("same address\n");
j=**&p;           /** same as j=*p (==i)      **/
  
```

- Referencing (&) and dereferencing (*) are inverse operations
 (*&x == x for any variable x)
- The referencing operator (&) can be applied to any type (even a pointer),
 but the dereferencing operator (*) can be applied only to pointer types



Referencing / dereferencing

Code example:

```

int      i = 3, j = 5,
        *p = &i, *q = &j, *r;

if(p == &i) printf ("same address\n");
j=**&p;           /** same as j=*p (==i)          **/
j = 7**p / *q+7;  /** same as j=7*i/j+7 (==14)     **/
  
```

- Ref and de-ref operators have high priority
- If **p** is of type **int***, then ***p** is of type **int** and can be an operand of an arithmetic operation

Referencing / dereferencing

Code example:

```

int      i = 3, j = 5,
        *p = &i, *q = &j, *r;

if(p == &i) printf ("same address\n");
j=**&p;           /** same as j=*p (==i)          **/
j = 7**p / *q+7;  /** same as j=7*i/j+7 (==14)      **/
*(r=&j) *= *p;     /** same as r=&j; j*=i;          **/
  
```

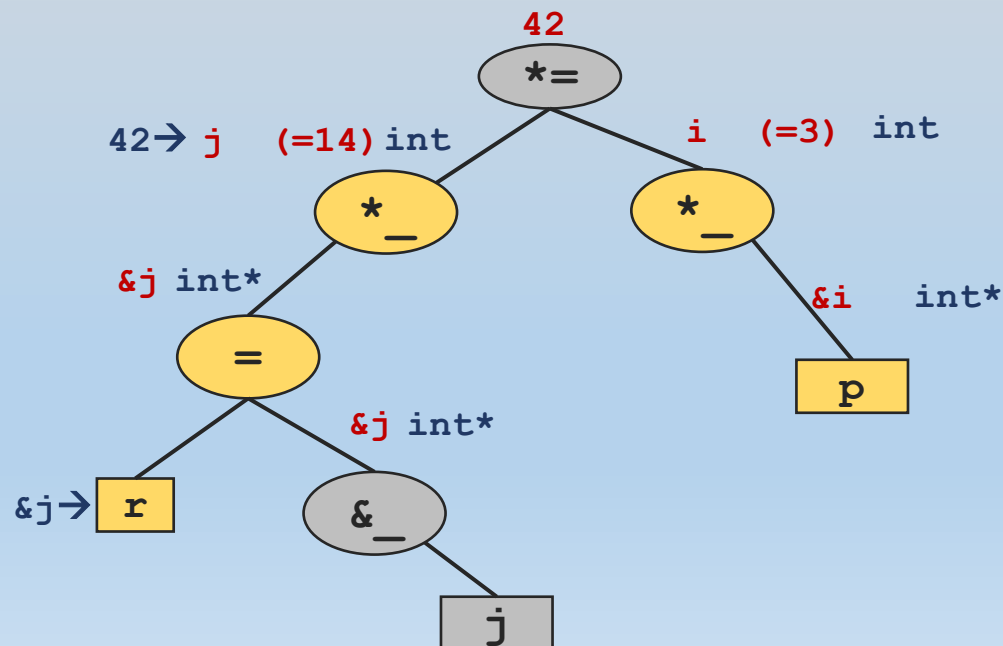
This line of code is put into an expression tree

int
int*

Example expression tree

Ref and deref operators can be used in complex expressions

```
* (r=&j) *= *p; /** same as r=&j; j*=i; */
```



Referencing / dereferencing

Code example:

```

int      i = 3, j = 5,
        *p = &i, *q = &j, *r;

if(p == &i) printf ("same address\n");
j=**&p;           /** same as j=*p (==i)           **/
j = 7**p / *q+7;  /** same as j=7*i/j+7 (==14)      **/
*(r=&j) *= *p;     /** same as r=&j; j*=i;          **/
  
```

- Any expression of pointer type can be dereferenced (even `r+2`)
- If a pointer is dereferenced at the left side of an assignment, then the **value stored at that address is changed**

Referencing / dereferencing

Code example:

```

int      i = 3, j = 5,
        *p = &i, *q = &j, *r;

if(p == &i) printf ("same address\n");
j=**&p;           /** same as j=*p (==i)          **/
j = 7**p / *q+7;  /** same as j=7*i/j+7 (==14)     **/
*(r=&j) *= *p;     /** same as r=&j; j*=i;         **/
p=&(i=3);          /** compilation error           **/
  
```

- You can only **reference a variable**, not a composite expression

error: invalid lvalue in unary '&'

Referencing / dereferencing

Code example:

```

int      i = 3, j = 5,
        *p = &i, *q = &j, *r;

if(p == &i) printf ("same address\n");
j=**&p;           /** same as j=*p (==i)      **/
j = 7**p / *q+7;  /** same as j=7*i/j+7 (==14)  **/
*(r=&j) *= *p;     /** same as r=&j; j*=i;      **/
p=&(i=3);          /** compilation error        **/
*i;               /** compilation error        **/
  
```

- You can **dereference any expression** that is of pointer type, but **only pointer types**

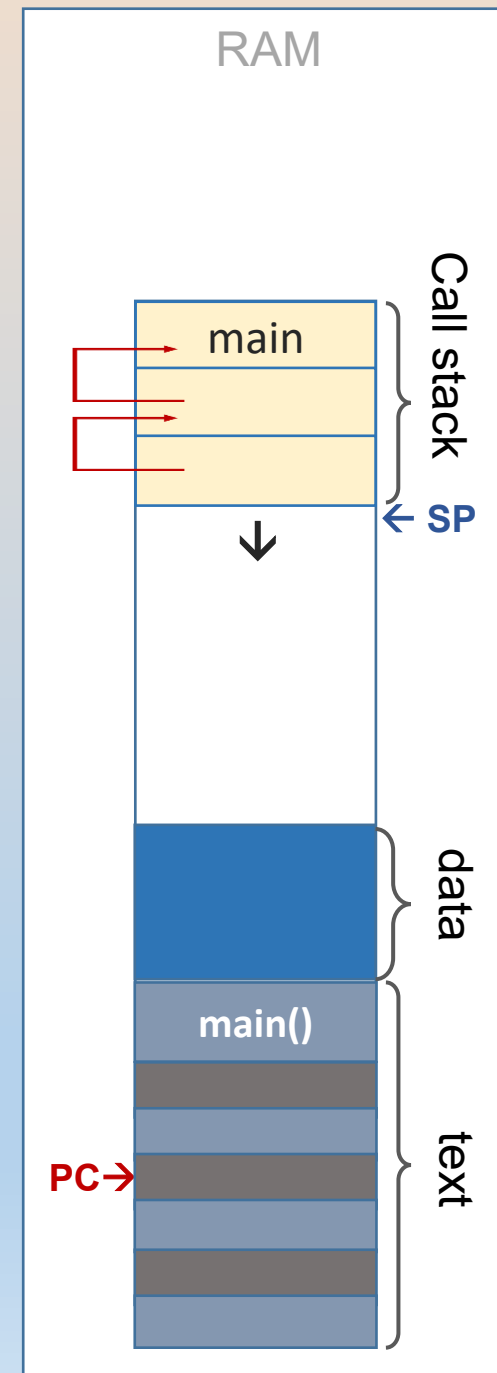
error: invalid type argument of 'unary *'

Referencing / dereferencing - summary

- Referencing (&) and dereferencing (*) are inverse operations
- The referencing operator (&) can be applied only to variables, but of any type (even a pointer)
- The dereferencing operator (*) can be applied to any expression as long as it is of pointer type
- If dereferencing is done on the right side of an assignment, then the content can be read
- If dereferencing is done on the left side of an assignment, then the content of that address can be modified

Recap

- A program has several **memory segments**
- Most variables are local and stored in **stack frames** of their invoked functions
- These variables are **automatically allocated** when function is called and **de-allocated** when function returns
- **Pointers allow us to pass access to local variables from one function to another**
- Pointers are a very powerful tool, but should be used with care [examples next week...]



At home...

- Homework exercise #4 due **Sunday, May 31 @ 21:00**
- Due to Shavuot, classes on Thursday and Friday are cancelled. Recitation on Friday will be made up with zoom sessions at night on Wednesday, 20 May and Sunday, 24 May. Check your email for details. Have a happy Shavuot!